

# Adaptive Acoustic Communications

## **Deliverable D1**

### Progress Report #1

v1.1  
May 15, 2026

Mandar Chitre  
Gabriel Chua



ARL, National University of Singapore

# Table of contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>3</b>  |
| 1.1      | Background . . . . .                                              | 3         |
| 1.2      | Scope of work . . . . .                                           | 3         |
| <b>2</b> | <b>Adaptive Communications</b>                                    | <b>5</b>  |
| 2.1      | Motivation . . . . .                                              | 5         |
| 2.2      | Protocol overview . . . . .                                       | 5         |
| 2.3      | Problem statement . . . . .                                       | 9         |
| 2.4      | Communication performance model . . . . .                         | 10        |
| 2.5      | OFDM performance model . . . . .                                  | 12        |
| 2.6      | Channel state information . . . . .                               | 15        |
| 2.7      | Model validation . . . . .                                        | 18        |
| 2.8      | Recommendation algorithm . . . . .                                | 21        |
| 2.9      | Next steps . . . . .                                              | 23        |
| <b>3</b> | <b>OFDM with Differentiable FEC</b>                               | <b>25</b> |
| 3.1      | Introduction . . . . .                                            | 25        |
| 3.2      | Residual Doppler . . . . .                                        | 25        |
| 3.3      | Conventional Defenses Against ICI . . . . .                       | 29        |
| 3.4      | JUNA: Coupled Estimation and Code-Constrained Detection . . . . . | 32        |
| 3.5      | Alternating Solver . . . . .                                      | 34        |
| 3.6      | Validation . . . . .                                              | 36        |
| <b>4</b> | <b>Summary and Next Steps</b>                                     | <b>48</b> |

# 1 Introduction

## 1.1 Background

Acoustic communications in underwater environments face significant challenges due to the complex and highly variable nature of acoustic propagation, multipath effects, and high ambient noise from natural and anthropogenic sources. These factors severely degrade signal quality and make it difficult to achieve reliable high-speed data transmission. Many commercial off-the-shelf (COTS) acoustic communications systems significantly underperform their published specifications in challenging underwater conditions. This project addresses these limitations by developing advanced acoustic communication techniques and adaptive communications protocols designed to maximize communication capacity in dynamic underwater environments.

## 1.2 Scope of work

This project builds upon specific work done in Project MUSIQUE where we developed several promising technology components that may be used for improving communication performance. We specifically focus on two tasks in this project:

### 1.2.1 Adaptive communications protocol

We aim to develop an adaptive communications protocol to optimize communication parameters automatically. The protocol will be integrated and demonstrated with Subnero modems to enable end-users to confidently deploy the modems in new environments without worrying about communication performance.

#### Key tasks:

1. Update the link tuning framework in UnetStack (software framework running on Subnero modems) to measure and track physics information.
2. Update the link tuning framework to continuously monitor link performance and adapt communication parameters during active data transmission, without the need for user intervention.
3. Refine and integrate physics-guided link adaptation algorithms on Subnero modems.

4. Demonstrate that the link adaptation algorithms achieve more robust communication at higher data rates in real-world scenarios in Singapore waters.

We report on progress on tasks 1, 2 and 3 in Chapter 2. Experiments will be undertaken in the next year to test and fine tune the protocol in Singapore waters.

### 1.2.2 Integrated channel estimation, equalization and decoding

Recent advances in differentiable computing made it possible to optimize channel estimation, equalization and decoding blocks jointly in a single algorithm. By combining differentiable forward error correction coding (DFEC) and differentiable channel estimation into a single deep learning framework, we aim to develop a unified and adaptive algorithm that can optimally communicate in a challenging channel.

#### Key tasks:

1. Extend work on differentiable FEC to time-varying channels and impulsive noise.
2. Develop a physics-guided acoustic time-varying channel estimation algorithm to guide channel equalization.
3. Combine differentiable FEC with physics-guided channel estimation and equalization to develop a completely differentiable communication system.
4. Test the algorithm in real-world scenarios in Singapore waters to demonstrate its practicality.

We report on progress on tasks 1, 2 and 3 in Chapter 3. Experiments will be undertaken in the next year to refine and test the communication algorithm in Singapore waters.

## 2 Adaptive Communications

### 2.1 Motivation

Subnero modems use UnetStack – an open-architecture underwater networking stack developed at ARL. Being software-defined, the modems support many modulation and forward error correction (FEC) techniques. Appropriate combination of techniques enable reliable communication in various kinds of channel conditions. When the channel is favorable, high-rate communication is made possible using coherent bandwidth-efficient modulation such as OFDM and a high-rate FEC. When the channel is challenging, reliable communication is enabled using incoherent modulation such as FH-BFSK combined with a low-rate FEC. By dynamically choosing modulation type, modulation parameters and FEC rate, we are able to adapt to a host of channel conditions, providing the best communication possible in those conditions. However, today, this adaptation requires human expertise, and isn't automated.

In project MUSIQUE, we developed techniques for automated adaptation of some of the communication parameters. However, the techniques require the modem to make a series of controlled measurements to inform link adaptation. Once the link is adapted to the channel, it can be used to communicate data. However, if the channel conditions change (as they inevitably do!), the link adaptation procedure has to be repeated. Ideally, we would want the link adaptation to happen continuously in the background, simply providing the user with the best possible link to use whenever he needs to send data. That is the goal of the work on *adaptive communication* in this project!

While the techniques for link adaptation from MUSIQUE form a good baseline to work from, they are primarily data-driven (with a little bit of physics encoded in them). For continuous adaptation, we often have very little data to work with. In this project, we therefore strive to add more physics to the link adaptation algorithm, with the hope that the physics provides sufficient constraints for rapid convergence that is required for continuous adaptation.

### 2.2 Protocol overview

In order to understand how continuous link adaptation would work, one has to first understand the flow of data in UnetStack. When an application wishes to transmit some information to another node, it encodes this information as a sequence of bytes and requests UnetStack to send a *datagram* containing those bytes as *payload* to the other node. Each node is identified

by a *node address*. The destination node address, along with some other metadata (e.g., reliability, priority, time-to-live, etc) is bundled together with the payload into a datagram. When UnetStack receives a request for datagram transmission, the sequence of events that take place depend on the contents of that datagram:

### Transport & routing layers

1. The encoding of the payload bytes by applications is not always efficient. In many cases, the same information can be represented using a smaller number of bytes. UnetStack uses content-type information and an array of available algorithms to *compress* the payload. The most general method for this is DEFLATE, which uses a combination of LZ77 and Huffman coding for compression. However, if content-type information is known, application-specific algorithms can also be used for compression. Irrespective of the algorithm, the payload can be replaced by the compressed payload (along with metadata required by the decompression algorithm) to reduce bandwidth requirements.
2. Most links have restrictions on the maximum number of bytes that can be transferred in one operation. This called a *maximum transfer unit* (MTU). Even when some links support large transfers, such transfers are not efficient. So links have a *recommended transfer unit* (RTU) which may differ from the MTU. If the datagram to transfer fits within the RTU (or the MTU), it can be queued for transmission. However, if it is too large, UnetStack *fragments* the datagram into smaller chunks that fit within the RTU and transmits those fragments. On the receiver side, these fragments are *reassembled* into the complete datagram to be delivered to the application.
3. In most cases, the destination node may be accessible over a 1-hop link. If that is not the case, however, UnetStack uses intermediate nodes to relay the information. If relaying is required, UnetStack encapsulates the original datagram to the final destination into another datagram for the immediate neighbor. The neighbor can extract the original datagram and relay it. At each hop, the optimal link parameters may be different, and decisions on which parameters to use would be made at each relay node. For simplicity of illustration, we will mostly assume that the destination is accessible over a 1-hop link, but the protocols described in this chapter are designed to work for multi-hop transfers as well (by being applied to each hop independently).
4. For unfragmented transfers with no reliability requirements, the datagram is handed over to the *data link* layer for best-effort delivery.
5. If the datagram had to be fragmented or the application requested reliability, the transport layer requires confirmation that each fragment was successfully delivered before the next fragment is sent. For single-hop transfers, this confirmation can be provided by the data link layer. For multi-hop transfers, single-hop confirmation from data link is not sufficient, and the transport agent has to generate end-to-end acknowledgements to confirm receipt of each fragment. If a fragment is lost, it is retransmitted after a timeout. If it can't be delivered after a predetermined number of retries, UnetStack eventually gives up and informs the application of the failure.
6. Eventually every application datagram request results in one or more datagram requests

at the data link layer. Each of these may vary in payload size and other metadata.

### Data link & physical layers

7. The physical layer has 2 logical communication channels. The CONTROL channel uses incoherent modulation with a low-rate FEC for robustness. It offers a fixed low data rate (typically ~100 bps) and a small MTU (typically 13 bytes). The DATA channel parameters are subject to adaptation, and may be adjusted to provide a high data rate and larger MTU when the channel conditions are favorable.
8. CONTROL and DATA channel acoustic transmissions are identified by separate acoustic preambles, but DATA channel parameters have to be synchronized on both end-points of a link in order for a received signal to be decoded correctly. In a network with more than 2 nodes, any node may initiate data transfer at any time, and so all nodes have to keep their DATA channels configured to a commonly agreed conservative set of parameters (not optimized for any specific link). This is akin to how maritime VHF voice communications work. All ships tune in to VHF channel 16 by convention. When one ship wants to communicate with another, it initiates the communication on channel 16. Then both ships agree to switch to another VHF channel for a conversation, but after that conversation is completed, both ships switch back to channel 16 to enable other ships to initiate communication when necessary.
9. In the special case of a network with only 2 nodes, however, both nodes may choose to keep their DATA channel on mutually agreed parameters that may be optimal for communication between them.
10. Requests to transmit a small datagram are serviced by sending the data using the CONTROL channel if the MTU is sufficient for the transfer, and DATA channel otherwise. Small datagrams are those that do not require further fragmentation-reassembly to fit within the physical layer MTU. Link adaptation is unnecessary for these, since the amount of data to transfer is too little to benefit from the adaptation.
11. Larger datagram requests require fragmentation. If the number of fragments is small, the data link layer uses erasure correction coding to add some redundancy (to guard against some loss) and transmits the sequence of fragments using the DATA channel. The sequence is preceded by a CONTROL *SETUP* frame containing information about the number of fragments. This frame could carry information on the DATA channel parameters, enabling the receiver to temporarily adopt those parameters before the fragments arrive. This allows for link-level adaptation, if desired, but requires the adaptation to be controlled by the transmitting node. Acoustic channel characteristics are not always reciprocal, and so the transmitter may not always have the necessary information for adaptation.
12. For even larger datagram requests, the data link layer uses erasure correction coding, but waits for the destination node to acknowledge (ACK) the SETUP frame before sending the fragments. This is to avoid wasting time transmitting all the fragments if the destination node wasn't able to hear the source node from the onset. The ACK frame can contain information on DATA channel parameters to use for the fragments,

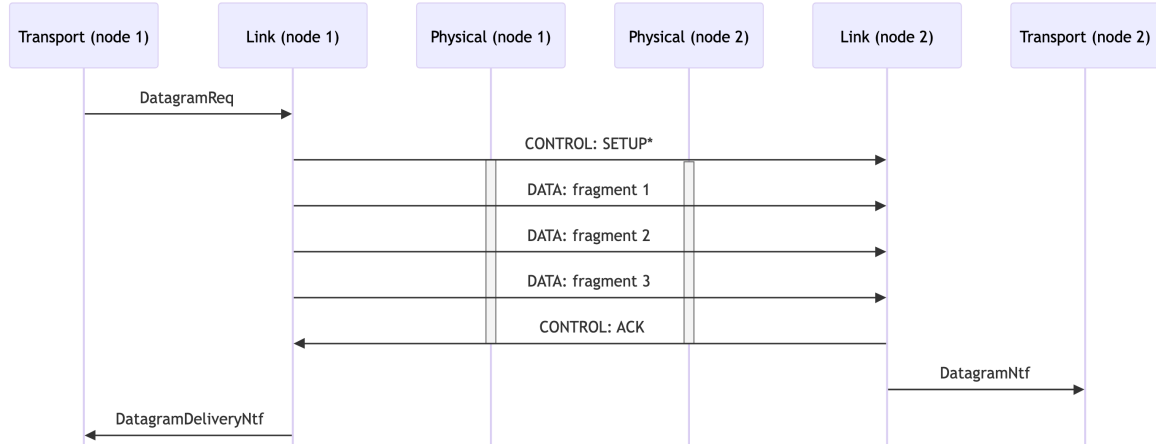


Figure 2.1: A sequence diagram showing the single-hop transfer of a mid-size datagram that fits within a single data link MTU but requires fragmentation at the data link layer into 3 physical layer fragments (including erasure correction redundancy). The SETUP\* frame contains adaptive DATA channel parameters to use for the transfer. The highlighted section on the physical layer timelines show the duration for the period of adoption of the DATA channel parameters on both nodes.

if desired, allowing the destination node to recommend optimal link parameters. The destination node can use the SETUP frame to assess channel characteristics to make its recommendations.

13. If the datagram request includes reliability, an ACK is sent by the destination node after receiving the fragments. If insufficient fragments were successfully received, the data link layer repeats the transfer with more fragments until the datagram is successfully delivered or the transfer is aborted (timeout / too many retries).

To summarize, small datagram transfers at a data link layer cannot benefit from link adaptation, except in the special case of a 2-node network. Mid-size datagrams can use link adaptation, but the link parameters have to be controlled by the transmitting node. This is sub-optimal in cases where the channel is not reciprocal. Large datagrams use a SETUP-ACK handshake that allows for a receiver-controlled link adaptation. While this might seem restrictive at first glance, it is *exactly* the large datagrams that benefit from the increased data rate via link adaptation. The overhead of adaptation is simply not worth it for small datagrams, since the overhead outweighs any benefit.

We illustrate the above two link adaptation scenarios in Figure 2.1 and Figure 2.2.

With this understanding of the overall protocol architecture within UnetStack, the key problem to solve is the algorithm for select the DATA channel parameters at each opportunity (transmission of SETUP\* or ACK\* frame). The selection has to be informed by channel state information (CSI), collected via channel measurements. However, as we will see in the next few sections,



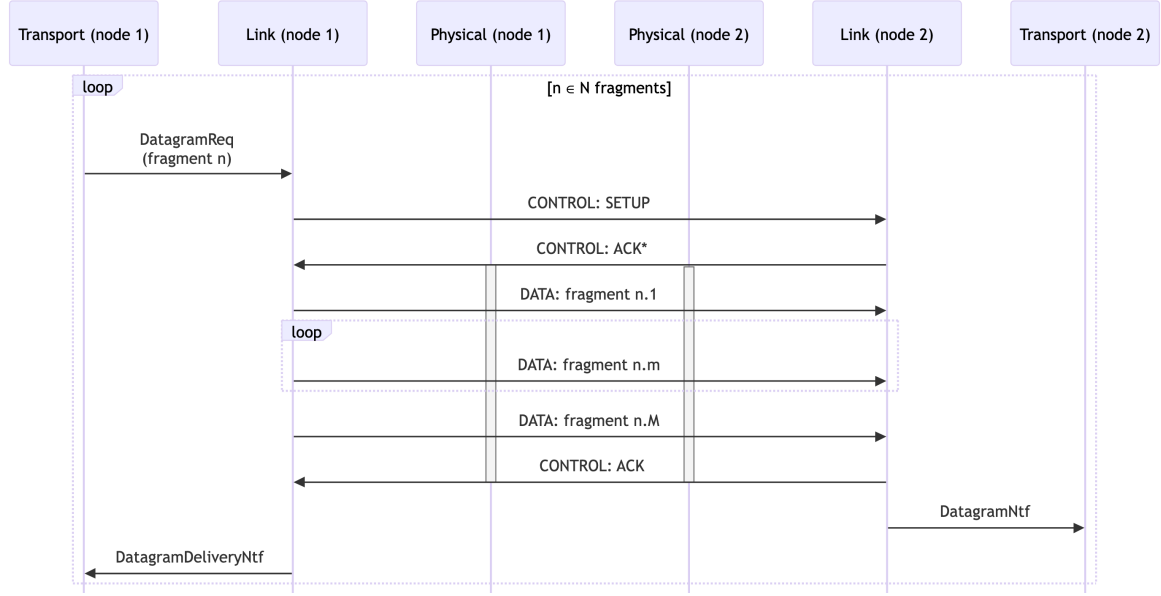


Figure 2.2: A sequence diagram showing the single-hop transfer of a large-size datagram. The ACK\* frame contains adaptive DATA channel parameters to use for the transfer. The highlighted section on the physical layer timelines show the duration for the period of adoption of the DATA channel parameters on both nodes. Each top-level fragment  $n$  may use a different set of adaptive DATA channel parameters.

we strive to make channel measurements opportunistically by observing acoustic receptions of normal CONTROL and DATA frames, avoiding additional special signal transmissions as far as possible.

## 2.3 Problem statement

Given CSI  $\xi$ , we wish to determine modulation parameters  $\theta$ , frame length  $N$  bits, and FEC code rate  $R$  to maximize the expected throughput for each data link transfer:

$$\max_{\theta, N, R} \Pi(\theta, N, R \mid \xi) D(\theta, N, R), \quad (2.1)$$

where  $\Pi(\theta, N, R \mid \xi)$  is the packet success rate (PSR) for the communication scheme defined by  $(\theta, N, R)$  operating in a channel defined by  $\xi$ , and  $D(\theta, N, R)$  is the data rate for that communication scheme.  $D(\cdot)$  is usually a deterministic function that is known, but  $\Pi(\cdot)$  can be difficult to estimate. In the next few sections, we will focus our attention to the estimation of PSR.

The above problem definition is rather abstract. We can make it more concrete in the context of Subnero modems. The key modulation parameters we wish to adapt are  $\theta \equiv (K, L, M, B, k_0)$ :

- $K \in \{2^7, \dots, 2^{13}\}$  is the number of carriers (modem parameter `phy[DATA].nc`)
- $L \in \mathbb{Z}_{\geq 0}$  is the cyclic prefix length (modem parameter `phy[DATA].np`)
- $M \in \{1, 2\}$  is the PSK modulation order (modem parameter `phy[DATA].psk`)
- $B \in (0, 1]$  is the fractional bandwidth (modem parameter `phy[DATA].bw`)
- $k_0 \in \{-\frac{K}{2}, \dots, \frac{K}{2}\}$  is the center carrier index (modem parameter `phy[DATA].dc0`)

While the Subnero modems support many other parameters, these are the key ones that control communication performance. We focus on tuning only these during continuous adaptation.

With these parameters, we can write down the expression for data rate:

$$D = \frac{NR}{n(K + L + L_s)/f_s + T_s}, \quad (2.2)$$

where  $f_s$  is the baseband sampling rate,  $L_s$  is the cyclic suffix length (`phy[DATA].ns`), and  $T_s$  is the time overhead from the detection preamble and synchronization pre- and postambles. The number of OFDM blocks  $n$  can be computed as:

$$n = \left\lceil \frac{N}{RM(\lfloor BK \rfloor - 1)} \right\rceil.$$

Furthermore, we restrict  $R \in \{\frac{2}{3}, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{5}, \frac{1}{6}\}$  for various LDPC FEC codes implemented in the Subnero modems.

## 2.4 Communication performance model

### 2.4.1 FEC performance model

In order to estimate PSR, we require a communication performance model. Let us first assume that we have a communication performance model  $b(\theta|\xi)$  that gives us the bit error rate (BER) of an uncoded differential OFDM system operating in a channel with CSI  $\xi$ . We focus our attention on a FEC performance model that converts this BER  $b$  into a PSR:

$$(b, N, R) \rightarrow \Pi$$

While there is no closed-form expression for LDPC to compute PSR from BER, we can use a finite block length *normal* approximation based on a binary symmetric channel (BSC) as an estimate:

$$\Pi = 1 - Q \left( (C(b) - R) \sqrt{\frac{N/R}{V(b)}} - \Delta \right), \quad (2.3)$$

with:

$$\begin{aligned} C(b) &= 1 + b \log_2 b + (1 - b) \log_2 (1 - b), \\ V(b) &= b(1 - b) \left( \log_2 \frac{1 - b}{b} \right)^2, \end{aligned}$$

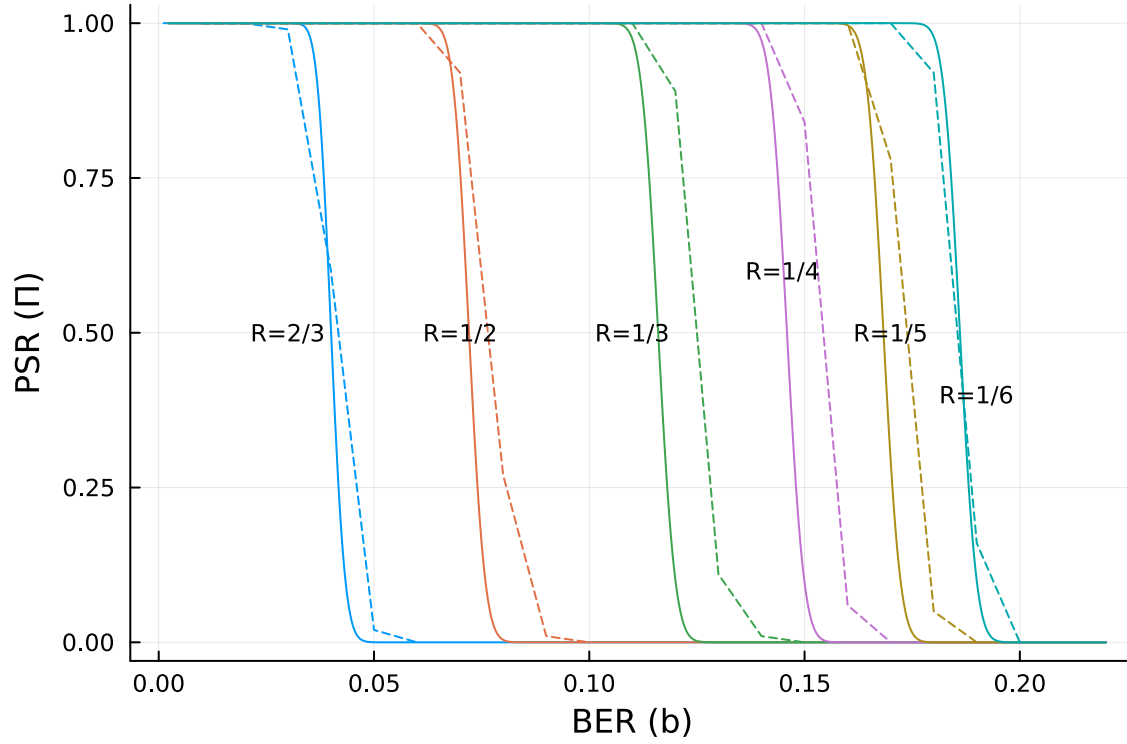


Figure 2.3: Modeled PSR against BER for various LDPC code rates  $R$  and  $N = 2048$  bits. The dashed lines are show measured performance using LDPC codes on Subnero modems. We see that the model closely matches the measurements, with the model being slightly conservative for some codes.

and:

$$Q(x) = \frac{1}{\sqrt{2\pi}} \int_x^\infty e^{-t^2/2} dt = \frac{1}{2} \operatorname{erfc} \left( \frac{x}{\sqrt{2}} \right).$$

The ideal expression uses  $\Delta = 0$ , but that yields over-optimistic PSR at high BER. We use a calibrated form with  $\Delta = 2 + 90b$  to get curves similar to ones we observe with the LDPC implementation in Subnero modems. We adopt this *calibrated* FEC model from here on.

Plots of PSR against BER for possible LDPC code rates  $R$  are shown in Figure 2.3. The figure also shows FEC performance measurements from Subnero modem, demonstrating a close match between model and reality. For some code rates, the model is somewhat conservative – this is intentional, as we would want the link adaptation to err on the side of safety.

## 2.5 OFDM performance model

### 2.5.1 Time-invariant channel

Now we shift our focus to determining  $b(\theta|\xi)$ . We begin with some simplifying assumptions:

1. The cyclic prefix length  $L$  is longer than the delay spread of the channel and hence we can assume per-carrier flat fading.
2. The channel does not vary significantly over the duration of one OFDM block, i.e., the channel is quasi-static. This allows us to ignore inter-carrier interference (ICI) due to loss of orthogonality in FFT.

We can select OFDM parameters such that the two assumptions are approximately true. In determining OFDM parameters, we may eventually need a performance model for what happens when the assumptions are violated, but we expect the violations to be small and so these assumptions provide a good starting point for our investigation.

We define a per-carrier effective SNR for frequency-differential BPSK detection:

$$\gamma_k \approx \frac{\text{Re}(H_k H_{k-1}^*)^2}{B\sigma_k^2 \sigma_{k-1}^2}, \quad (2.4)$$

where  $\sigma_k^2$  is the noise variance for carrier  $k$  and  $H_k$  is the frequency-domain channel estimate:

$$H_k = \int_{-\infty}^{\infty} h(\tau) e^{-i2\pi f_k \tau} d\tau, \quad (2.5)$$

with  $f_k = kf_s/K$ .

Per-carrier BER can be computed from SNR:

$$b_k = \frac{1}{2} e^{-\gamma_k}. \quad (2.6)$$

For differential QPSK with Gray code mapping (as we use in the Subnero modems), we can use:

$$\begin{aligned} \gamma_k &\approx \frac{\text{Re}(H_k H_{k-1}^*)^2}{B\sigma_k^2 \sigma_{k-1}^2 + |H_k|^2 \sigma_k^2 + |H_{k-1}|^2 \sigma_{k-1}^2}, \\ b_k &\approx Q\left(\sqrt{\frac{\gamma_k}{2}}\right). \end{aligned} \quad (2.7)$$

The QPSK version isn't as accurate as the BPSK version, as the differential QPSK BER does not have a nice closed form expression. However, it is a slightly conservative approximation that works well in practice.

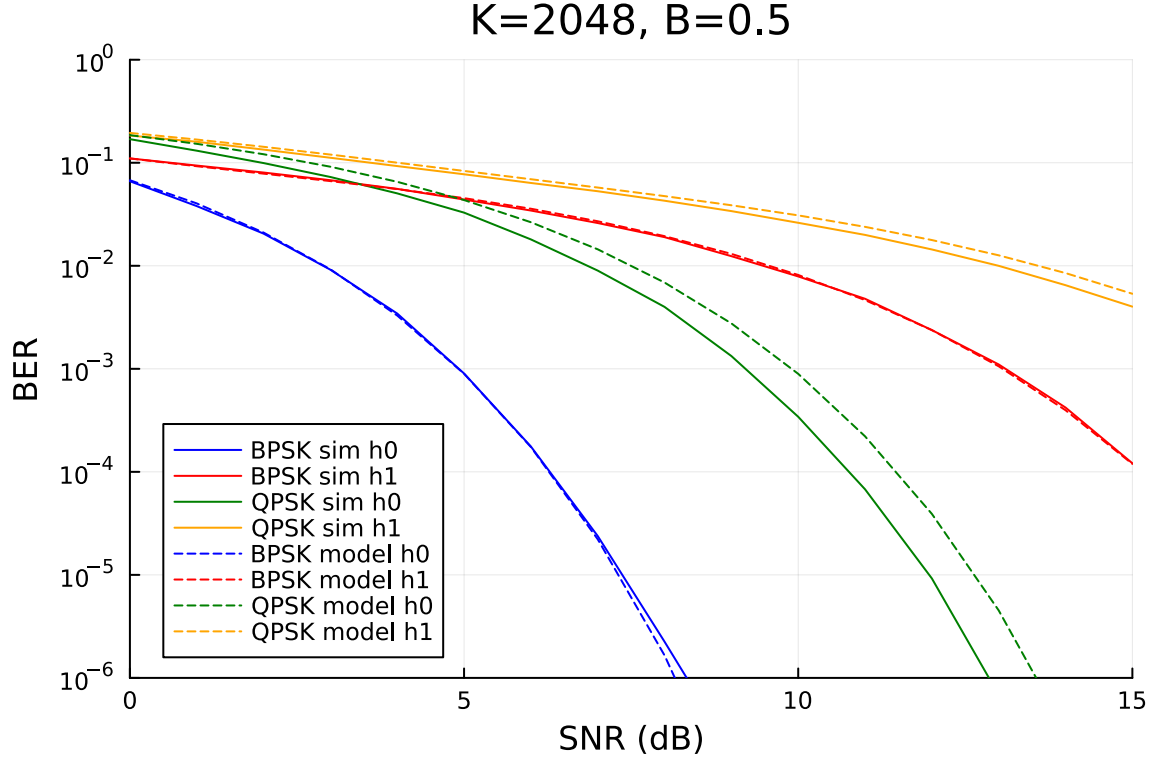


Figure 2.4: Comparison between simulation and performance model for OFDM with differential BPSK and QPSK with two different channels (h0 and h1). Channel h0 is an AWGN channel for benchmarking. Channel h1 is a time-invariant 3-ray 10-tap frequency selective channel. We see a good match between theory and simulation.

The average BER is computed from all the per-carrier BERs directly:

$$b = \frac{1}{2\bar{K} + 1} \sum_{k=-\bar{K}}^{\bar{K}} b_{k+k_0}, \quad (2.8)$$

with  $\bar{K} = \lfloor BK/2 \rfloor$ .

To use this model, we require the channel impulse response (CIR)  $h(\tau)$  and noise measurements  $\sigma_k^2$  to be available in CSI  $\xi$ .

To test the model, in Figure 2.4, we compare the model against simulation results for two benchmark time-invariant channels:

- h0: Additive white Gaussian noise (AWGN) channel.
- h1: 3-ray 10-tap channel:  $[1, 0, 0, -0.7, 0, 0, 0, 0, 0, 0.3]$ , representing a typical shallow water frequency selective channel.

We see a good match between simulation and the model. The QPSK AWGN case slightly over-estimates the BER, making the model slightly conservative in simple channels.

### 2.5.2 Time-varying channel

While assumption 1 in the previous section is easy to satisfy by choosing a  $L$  based on the impulse response from the CSI, assumption 2 can be violated if the chosen  $K$  is larger than the coherence time of the channel. The model will predict low BER as  $K$  increases, but an OFDM scheme with large  $K$  will perform poorly in practice. We need to modify the model to correct this.

The time-variability of the channel can be characterized in terms of the  $\tau$ -lag auto-correlation of the channel's frequency response over the band of interest. If the correlation between the channel measured  $\tau$  seconds apart is  $\rho(\tau)$ , we can include its impact on BER by using a modified per-carrier effective SNR to include the effect of ISI:

$$\gamma'_k = \frac{(1 - \alpha)\gamma_k}{1 + \alpha\gamma_k},$$

where

$$\alpha = 1 - \left| \rho \left( \frac{K}{f_s} \right) \right|^\beta,$$

for  $1 \leq \beta \leq 2$ . When  $\beta = 1$ , this expression provides a weak penalty for channel variation. At the other extreme,  $\beta = 2$  provides a strong penalty.

In the terrestrial wireless literature, the time variability of a channel is often characterized by a Jakes Doppler model with parameter  $f_D$ . The Doppler parameter  $f_D$  has a nice physical interpretation as the Doppler spread of the channel (typically measured in Hz). If this can be measured or estimated as part of CSI  $\xi$ , we can compute  $\rho(\tau)$  from  $f_D$ :

$$\rho(\tau) = J_0(2\pi f_D \tau),$$

where  $J_0(\cdot)$  is the 0<sup>th</sup> order Bessel function. For OFDM to work well, we require  $K f_D \ll f_s$ . In that case, we can use a first-order approximation to measure the correlation over one symbol period:

$$\rho \left( \frac{K}{f_s} \right) \approx 1 - \left( \frac{\pi K f_D}{f_s} \right)^2.$$

By including this correction to SNR, we can observe (Figure 2.5) the expected U-shaped curve for  $K$ , with too low a  $K$  being penalized as carrier spacing becomes more than the coherence bandwidth and too high a  $K$  being penalized as the OFDM block becomes longer than the coherence time of the channel.

The key question we need to answer is whether the Jakes Doppler model is a good model for underwater channels. To answer this, we extract channel estimates from underwater channel

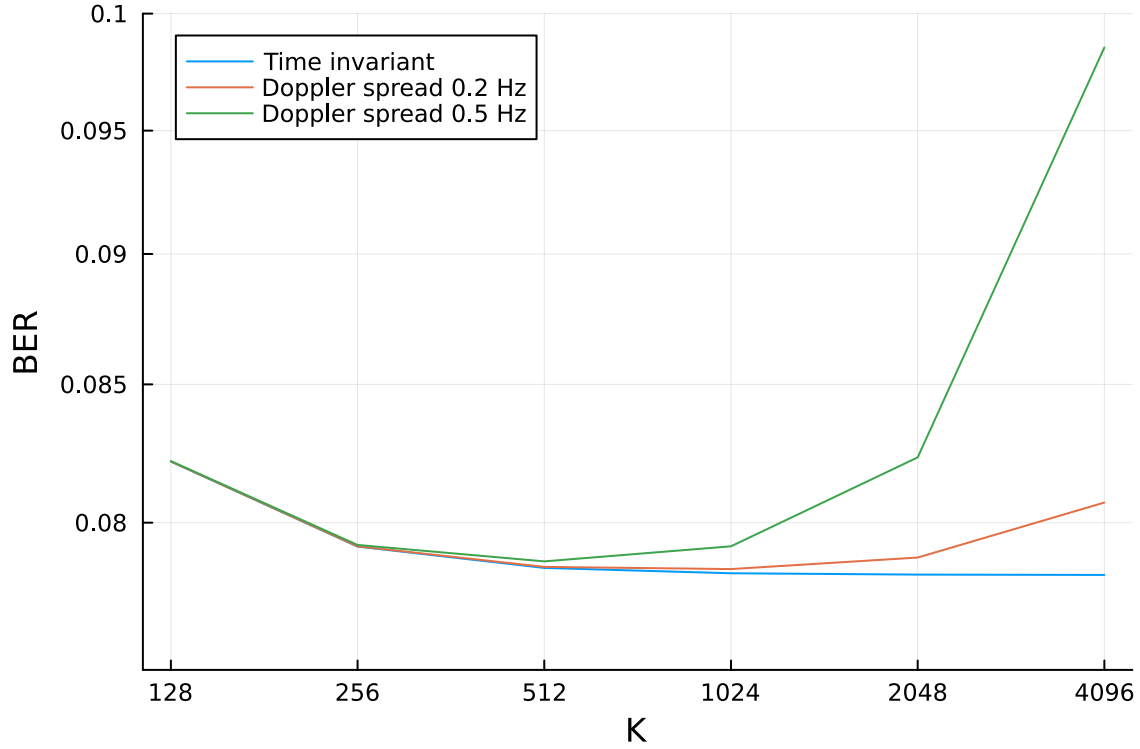


Figure 2.5: Typical U-shaped curve for  $K$  for  $\beta = 1$ . The time-invariant model predicts improvement in BER as  $K$  increases. Time variability severely limits the maximum usable  $K$ .

measurements made in Singapore waters (near St Johns island in 2024). The time-varying impulse response of the channel is shown in Figure 2.6. Figure 2.7 shows the measurements against a Jakes Doppler model (single parameter  $f_D$ ) and an empirical model (2-parameters):

$$\rho(\tau) = 1 + \nu_1 \tau + \nu_2 \tau^2.$$

We see that the 2-parameter empirical model is a much better fit. The empirical model is closely related to the Jakes Doppler model – it simplifies to the Jakes Doppler first-order approximation for  $\nu_1 = 0$  and  $\nu_2 = -\pi^2 f_D^2$ . We can therefore adopt the empirical model without any loss of generality, fitting a Jakes Doppler model as a special case when the time-variability matches it.

## 2.6 Channel state information

As we saw in the previous section, CSI plays a critical role in estimating OFDM performance. At the very least, CSI  $\xi$  must contain:

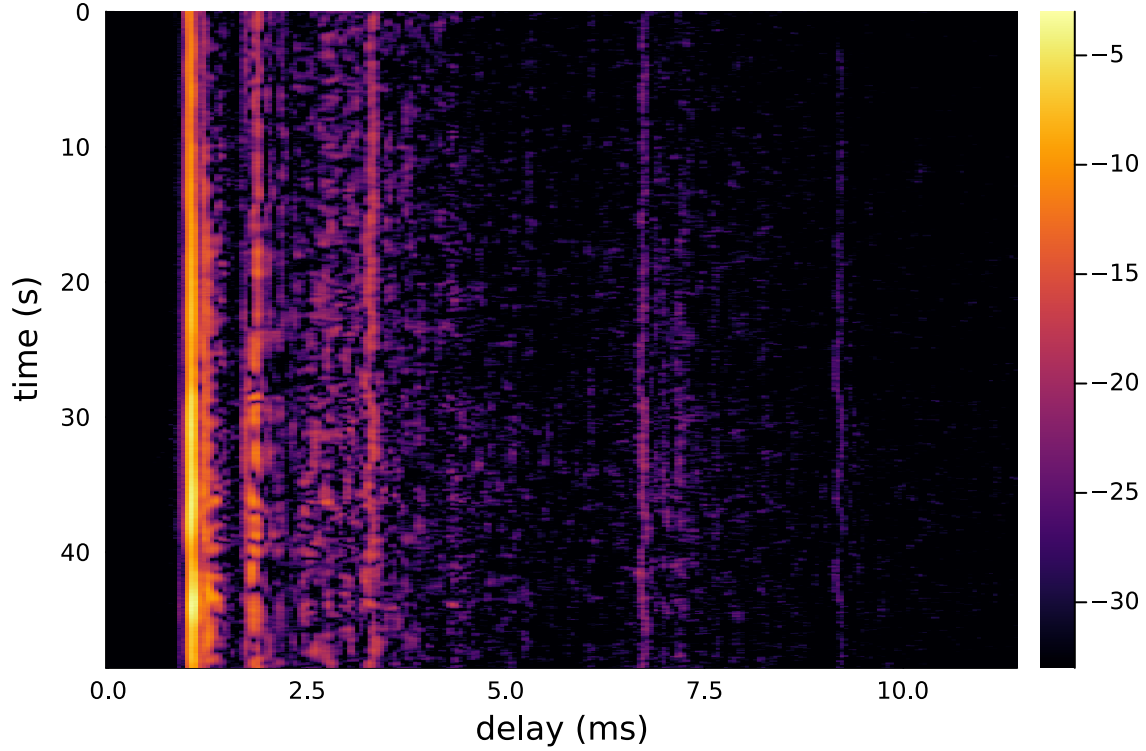


Figure 2.6: Measured time-varying channel impulse response during the 2024 St Johns island experiment.

- Estimate of CIR  $h(\tau)$ .
- Noise estimate  $\sigma_k^2$ .
- Time-variability parameter estimates  $\nu_1$  and  $\nu_2$ .

We introduce a `csi` agent in UnetStack to collate CSI by opportunistically observing incoming frames from peer nodes. In this section, we outline how each of the above estimates are obtained.

### 2.6.1 Channel impulse response estimation

Every frame in UnetStack has a preamble. The default preambles for CONTROL and DATA frames are hyperbolic frequency modulated (HFM) up- and down-sweeps respectively. These preambles have good autocorrelation properties, and are ideal for CIR estimation by matched filtering.

Given a received signal  $x(t)$  and a preamble  $s(t)$  that is contained in the received signal, a CIR



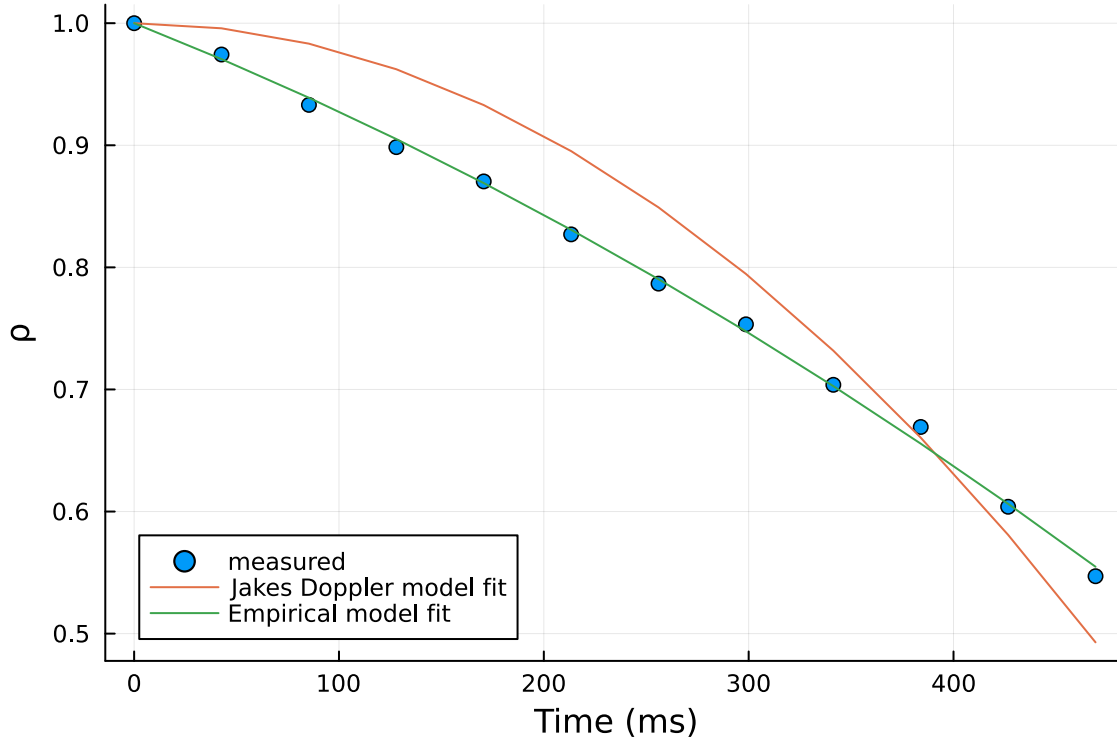


Figure 2.7: Measured channel auto-correlation  $\rho(\tau)$ , along with 2 model fits – 1-parameter Jakes Doppler model and 2-parameter empirical model.

estimate is obtained as follows:

$$h(\tau) = \int_{-\infty}^{\infty} s^*(t - \tau)x(t)dt.$$

While the limits on integration are infinite,  $x(t)$  is finite and so the contributions to the integral have a finite support.

### 2.6.2 Noise estimation

Noise can be estimated from recordings when no signal arrives from any peer. In practice, `csi` continuously records noise samples. When a frame is detected, noise samples spanning the duration of that frame are discarded. Given recorded noise  $x(t)$ , noise variance is estimated by taking FFTs of noise samples and averaging:

$$\sigma_k^2 = \lim_{T \rightarrow \infty} \frac{1}{2T} \left| \int_{-T}^T x(t) e^{-i2\pi f_k t} dt \right|^2 \approx \frac{1}{J} \sum_{j=1}^J |\mathcal{F}_K \mathbf{x}_j|^2,$$

where  $\mathcal{F}_K$  is a Fourier matrix of size  $K$  and  $\mathbf{x}_j$  is the  $j^{\text{th}}$   $K$ -sample block of recorded noise.

### 2.6.3 Time-variability model parameters estimation

The time-variability parameters  $\nu_1$  and  $\nu_2$  can be estimated from a successfully received OFDM frame with at least 3 OFDM blocks (numbered 0, 1, ...,  $N_b$ ). Consider blocks 0 and  $\tau$ . Since the frame was successfully received, we know transmitted symbols  $X_{0k}$ . We can get estimates of the channel at each of the two blocks:

$$H_{0k} = \frac{Y_{0k}}{X_{0k}},$$

$$H_{\tau k} = \frac{Y_{\tau k}}{X_{\tau k}},$$

where  $Y_{0k}$  and  $Y_{\tau k}$  are the received symbols in blocks 1 and  $\tau$  respectively. We next estimate a lag- $\tau$  correlation from the channel estimates:

$$\rho_\tau = \left| \frac{\mathbf{H}_0^H \mathbf{H}_\tau}{\|\mathbf{H}_0\| \|\mathbf{H}_\tau\|} \right|.$$

Assuming the empirical model from Section 2.5.2, we have:

$$\rho_\tau = 1 + \nu_1 \tau + \nu_2 \tau^2.$$

We can obtain  $\nu_1$  and  $\nu_2$  by solving a simple linear system of equations:

$$\begin{pmatrix} \rho_1 \\ \rho_2 \\ \vdots \\ \rho_{N_b} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 2 & 4 \\ \vdots & \vdots \\ N_b & N_b^2 \end{pmatrix} \begin{pmatrix} \nu_1 \\ \nu_2 \end{pmatrix}.$$

## 2.7 Model validation

In deriving the OFDM performance model, we made several assumptions, simplifications and approximations. This enabled us to make the mathematics tractable, but potentially at the cost of losing some accuracy. We first explore how well the model compares against time-invariant OFDM simulations. We then compare the model predictions against experimental OFDM testing using direct channel replay.

We generate 1000 random OFDM communication schemes with  $K$  from 128 to 8192 carriers,  $B$  from 0.2 to 0.5,  $k_0$  chosen to place the OFDM signal randomly within the band of interest, and  $M$  of 1 (BPSK) or 2 (QPSK). We set  $L$  to be large enough to cover the delay spread. We simulate OFDM performance in a synthetic 3-ray 10-tap channel (h1 from Section 2.5.1) for SNR between 0 to 10 dB. We predict the BER performance for each OFDM scheme for the same channel and SNR and compare it with the measured BER from simulations. The results are shown in Figure 2.8. We see a very good match between our model and measurements.

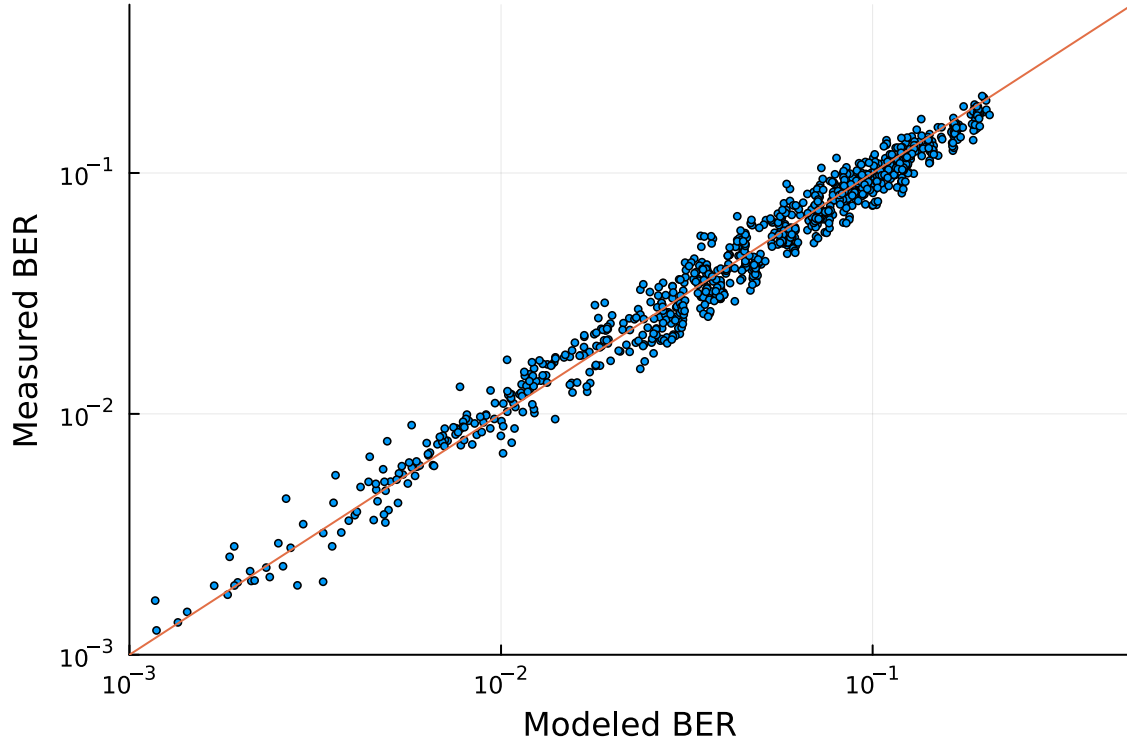


Figure 2.8: Modeled v/s measured BER for random OFDM schemes in a simulated time invariant channel with varying SNR between 0–10 dB.

While the good match between model and simulation is encouraging, it does not guarantee a good match between model and reality. The simulation is time invariant, whereas real channels are time varying. Our model handles time variation, so we next test the model against a OFDM performance in a time-varying channel. While at-sea experiments provide a real channel, it is impossible to measure the performance of a large number of OFDM schemes in the same channel. Real channels not only exhibit the fast time-varying which we are interested in modeling, but also exhibit slow time-variation that is a nuisance for our analysis. Direct channel replay simulations provide an alternative to at-sea OFDM experiments, where an at-sea experiment records an estimate of a time-varying channel, and later that estimate can be used to conduct simulations that include the effect of time variability as observed during the experiment. We therefore use direct channel replay based on the 2024 St Johns island experiment discussed in Section 2.5.2 (see Figure 2.6).

We follow the same methodology of generating 1000 random OFDM schemes with  $K$  from 256 to 8192 carriers,  $B$  from 0.2 to 0.5,  $k_0$  chosen to place the OFDM signal randomly within the band of interest, and  $M$  of 1 (BPSK) or 2 (QPSK). We set  $L$  to be large enough to cover the measured delay spread from the 2024 St Johns island experiment. We then measure OFDM performance using the St Johns island replay channel. We predict the BER performance for

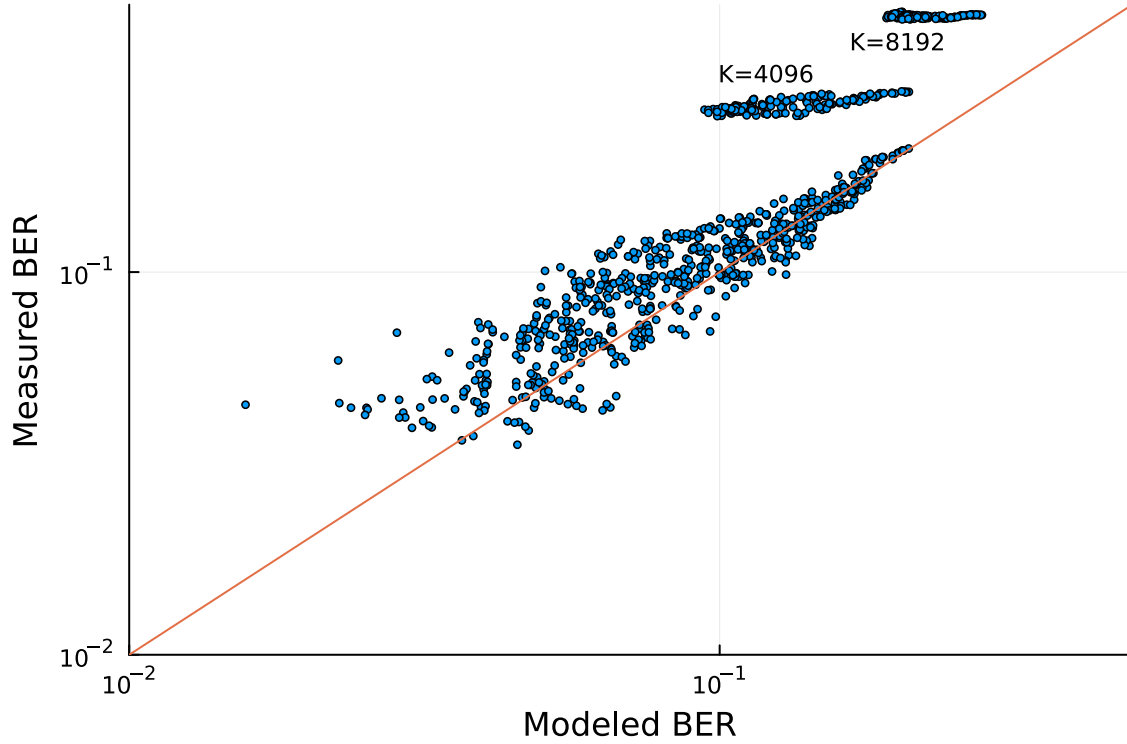


Figure 2.9: Modeled v/s measured BER for random OFDM schemes in the St Johns island replay channel.

each OFDM scheme based on CSI extracted from the replay channel using the methodology outlined in Section 2.6. Since we cannot control the SNR in the replay channel directly, we add a fixed level of noise to the replay output and estimate the average resulting SNR. We use this SNR to predict performance using our model and compare it with the measured BER from direct replay. The results are shown in Figure 2.9. We see a good match between our model and measurements, except for two sets of outliers at  $K = 4096$  and  $K = 8192$ . For these sets, the model underpredicts the BER by roughly a factor of 2.

To understand this dependence of BER accuracy on  $K$ , we fix  $B = 0.4$ ,  $k_0 = 0$  and  $M = 2$  and only vary  $K$  while measuring the BER via direct replay and predicting it using the model with  $\beta = 2$ . We plot both in Figure 2.10. We see a good match for  $K \leq 2048$ , but a gap at higher  $K$ . This is consistent with the observation from Figure 2.9 where the BER for  $K = 4096$  and  $K = 8192$  was underpredicted.

Recall from Section 2.5.2 that the time-variability model was only valid when  $K$  was small enough for OFDM to work well. When  $K$  is too large, the model predicts that the OFDM performance will degrade, but in reality, OFDM fails even more catastrophically than the prediction. In practice, we are only interested in accurately modeling OFDM performance

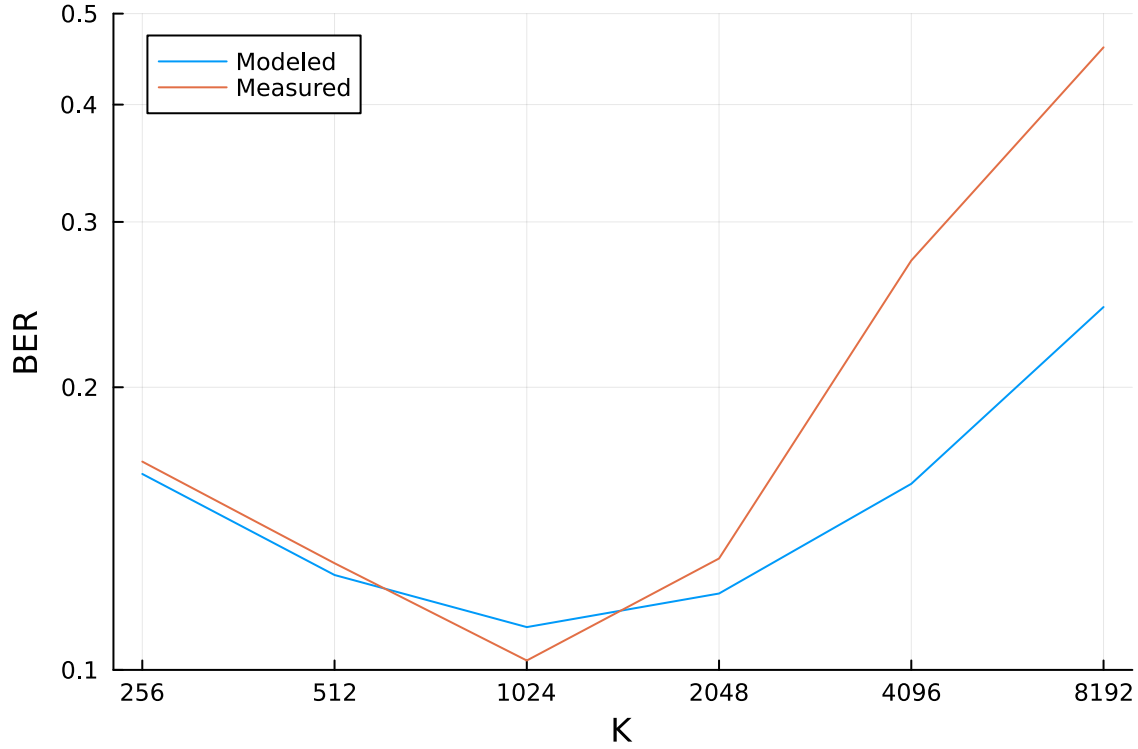


Figure 2.10: Comparison of measured and modeled BER for varying  $K$  in the St Johns island replay channel. Other OFDM parameters are kept fixed at  $B = 0.4$ ,  $k_0 = 0$  and  $M = 2$ .

when OFDM is likely to work; when OFDM fails, it is sufficient for us to predict that it would fail, even if we don't predict the BER accurately. We therefore choose not to add complexity to our model to handle this high-BER regime accurately, as the performance of the current model to predict BERs up to about 15% accurately is sufficient for our needs.

## 2.8 Recommendation algorithm

We now have all the pieces in place to write down the algorithm for recommending a set of OFDM parameters  $\theta$  ( $K$ ,  $L$ ,  $M$ ,  $B$  and  $k_0$ ), frame length  $N$ , and code rate  $R$ , given CSI  $\xi$  ( $h(\tau)$ ,  $\sigma_k^2$ ,  $\nu_1$ ,  $\nu_2$ ).  $\sigma_k^2$  is measured from a noise measurement when the channel is idle.  $h(\tau)$  can be estimated from any transmission from the peer node.  $\nu_1$  and  $\nu_2$  require an OFDM transmission from the peer node (a FH-BFSK CONTROL frame is not sufficient). When bootstrapping a data transfer, if a OFDM frame isn't available for estimation, our algorithm has to handle the possibility that  $\nu_1$  and  $\nu_2$  may not be available.

### 2.8.1 Link adaptation algorithm

The recommended algorithm is sequential with the following steps:

1. **Determine  $L$ .** Pick minimum  $L$  such that:

$$\frac{\int_0^{L/f_s} |h(\tau)|^2 d\tau}{\int_0^\infty |h(\tau)|^2 d\tau} \geq \eta_L,$$

for some threshold  $\eta_L$  (e.g., 0.95).

2. **Determine  $K$ .** From the set of allowable values, pick  $K$  to minimize  $b$  for  $M = 1$  and nominal values of  $B$  and  $k_0$  (e.g.,  $B = 0.4$ ,  $k_0 = 0$ ) assuming CSI contains estimates for  $\nu_1$  and  $\nu_2$ . If these estimates are not available, pick the minimum  $K$  that yields  $b$  that is not significantly more than  $b$  for the largest allowable  $K$ . This yields a conservative estimate for  $K$ , which can be refined later once  $\nu_1$  and  $\nu_2$  become available after the first OFDM transmission during data transfer.
3. **Determine  $B$ ,  $k_0$ ,  $M$ ,  $R$  and  $N$ .** Perform a grid search over all possible  $B$  (quantized as bandwidth can only be adjusted by increasing/decreasing a single carrier),  $k_0$  within feasible operating band,  $M$  and  $R$  to maximize data rate:

$$\max_{B, k_0, M, R} \Pi(\theta, N, R \mid \xi) D(\theta, N, R),$$

with the largest  $N$  selected such that the duration of the frame is less than some  $T_{\max}$  (e.g., 1 second). First, we compute  $n$ :

$$n = \left\lfloor \frac{T_{\max} - T_s}{K + L + L_s} f_s \right\rfloor,$$

and then:

$$N = 8 \left\lfloor \frac{nRM(\lfloor BK \rfloor - 1)}{8} \right\rfloor.$$

This ensures that  $N$  is an integer number of bytes, since UnetStack frames are composed of bytes. When performing the grid search, it is recommended that the BER from the OFDM performance model be inflated by a tunable factor  $\zeta$  (typically between 1.5 to 3, with 1.5 being aggressive and 3 being conservative) to account for the fact that the actual OFDM performance varies around the OFDM performance model prediction.

4. **Adjust  $R$  and  $N$**  when more information becomes available. If a replay channel is available (constructed by a probe or OFDM transmission from peer node), use it to determine BER for the chosen OFDM scheme. Adjust  $R$  and  $N$  according to the algorithm in step 3, but using measured BER from direct replay instead of the inflated BER from the OFDM performance model. Since direct replay BER is stochastic, we recommend running the replay multiple times and taking the maximum measured BER to determine

*R.* Once a successful transmission is made in the water, actual BER can be measured. If such a measurement is available, it should be used in preference to the direct replay BER or the modeled BER. If multiple BER estimates are available, we recommend using a trimmed-maximum as a conservative BER estimate.

## 2.8.2 Testing using channel replay

To test the above algorithm, we use the 2024 St John island replay channel from Section 2.5.2 (Figure 2.6) to extract CSI using the procedure described in Section 2.6. We use that with the above algorithm (steps 1–3) to recommend an OFDM scheme:

OFDM parameters:

$K=1024$ ,  $L=96$ ,  $B=0.5$ ,  $k_0=0$ ,  $M=2$ ,  $R=1/3$ ,  $N=7840$

Uncoded BER: 0.077

Expected data rate: 6835 bps

In order to test the recommendation, we transmit 100 frames using the recommended OFDM scheme via the replay channel and measure throughput:

Average uncoded BER: 0.094

Packet success rate: 1.0

Throughput: 6835 bps

This shows that the recommended scheme indeed works as expected in the replay channel.

## 2.9 Next steps

We now have a physics-based algorithm that is able to recommend OFDM schemes based on opportunistic CSI. The algorithm was tested using a replay channel from Singapore waters, and shown to perform well. The next steps include:

1. **Test the algorithm on a larger set of replay channels.** We have access to a channel repository from replay channels from many parts of the world, which we can use to test the algorithm.
2. **Fine tune the algorithm.** There are several tunable hyper-parameters in the algorithm that control its aggressiveness. These can be fine tuned using the channel repositories for testing.
3. **Implement the algorithm on the modem.** Many parts of the algorithm have already been implemented on the Subnero modem, but some parts still need integration.

4. **Test using VAO.** The virtual acoustic ocean (VAO) is an underwater acoustic modeling framework that can be used with Subnero modems to transmit data through simulated channels. VAO supports replay channels and can be used with the channel repository for testing. The modem implementation can be tested to ensure that it yields the expected performance in the replay channels.
5. **Test at sea.** The final test of the algorithm will be at sea. We plan on more than one experiments to test the algorithm in several different channel conditions.



## 3 OFDM with Differentiable FEC

### 3.1 Introduction

Orthogonal frequency-division multiplexing (OFDM) relies on the channel changing slowly enough that the frequency-domain input–output relation is approximately diagonal. Under residual Doppler this assumption breaks down: even after packet acquisition and coarse Doppler correction, the channel varies within the OFDM interval, subcarrier orthogonality is destroyed, inter-carrier interference (ICI) emerges, and pilot-only equalization fails. The receiver must then estimate reliable *pre-decoder soft symbols* — the soft subcarrier estimates supplied to the LDPC decoder — under structured intra-symbol time variation.

This report develops an improved OFDM receiver in four steps. First, we establish the CP-OFDM baseline without Doppler or platform motion, then show how residual Doppler converts the diagonal operator into a banded one (Section 3.2). Second, we survey conventional defenses against this breakdown — comb pilots, partial-FFT temporal slicing, and the RLS-based Partial-FFT Combiner (RPC) — and identify the point at which separating RPC from LDPC decoding starts to hurt (Section 3.3). Third, we introduce *JUNA* — a coupled formulation in which soft-symbol estimation and LDPC-constrained decoding share a single objective so that pilots, channel structure, and code constraints regularize one another throughout inference (Section 3.4). The remaining sections present an alternating solver (Section 3.5) and a packet-level validation across a population of underwater acoustic channels (Section 3.6).

The joint approach adapts principles from differentiable forward error correction (DFEC) from ISI channels to Doppler-distorted OFDM, and is most useful where local pilot information alone is insufficient: sparse pilots, strong residual Doppler, and mid-to-low SNR where decision-directed updates become unreliable.

### 3.2 Residual Doppler

#### 3.2.1 Pristine CP-OFDM and the Diagonal Model

Table 3.1: Notation used in this chapter.

| Symbol             | Meaning                                                            | Symbol             | Meaning                                          |
|--------------------|--------------------------------------------------------------------|--------------------|--------------------------------------------------|
| $N$                | number of OFDM subcarriers                                         | $\mathcal{P}(k)$   | nearby pilot set used for carrier $k$            |
| $N_{\text{cp}}$    | cyclic prefix length                                               | $\delta$           | ridge parameter in the RPC fit                   |
| $\mathcal{K}$      | active-carrier index set                                           | $\mathbf{U}_k$     | stacked pilot observation matrix for carrier $k$ |
| $\mathcal{P}$      | pilot-carrier index set                                            | $\mathbf{d}_k$     | stacked pilot symbol vector for carrier $k$      |
| $\mathcal{D}$      | data-carrier index set                                             | $\mathbf{R}_k$     | pilot correlation matrix                         |
| $X_k$              | transmitted symbol on carrier $k$                                  | $\mathbf{r}_k$     | pilot cross-correlation vector                   |
| $Y_k$              | received symbol on carrier $k$                                     | $\beta$            | forgetting factor in the RLS recursion           |
| $\widehat{H}_k$    | pilot-estimated channel frequency response                         | $\widehat{X}_k$    | one-tap equalized symbol using $\widehat{H}_k$   |
| $L_h$              | time-domain channel-estimation length                              | SFM                | spectral flatness measure                        |
| $\rho_{\text{LS}}$ | ridge weight in pilot-LS channel estimation                        | $D_{\text{notch}}$ | max-to-min frequency-response notch depth        |
| $H_{k\ell}(a)$     | fixed-Doppler frequency-domain coupling from carrier $\ell$ to $k$ | $\bar{\eta}$       | mean ICI ratio from $H_{k\ell}(a)$               |
| $\nu_k$            | frequency-domain additive noise on carrier $k$                     | $\mathbf{W}$       | collection of all per-carrier combining vectors  |
| $\nu_{\text{t}}$   | time-domain additive noise vector                                  | $\mathbf{z}$       | bit-level latent optimization variables          |

Consider an OFDM system with  $N$  subcarriers and cyclic prefix of length  $N_{\text{cp}}$ , pilot set  $\mathcal{P}$  and data set  $\mathcal{D}$ . Let  $\mathbf{X} = [X_0, \dots, X_{N-1}]^T \in \mathbb{C}^N$  denote one frequency-domain OFDM symbol and  $\mathbf{F}$  the unitary  $N \times N$  DFT matrix. The useful time-domain block is  $\mathbf{x} = \mathbf{F}^H \mathbf{X}$ . After CP insertion, transmission through an  $L$ -tap channel with  $L - 1 \leq N_{\text{cp}}$ , and CP removal, the received block satisfies:

$$\mathbf{r} = \mathbf{H}_{\text{cir}} \mathbf{x} + \nu_{\text{t}}, \quad (3.1)$$

with  $\mathbf{H}_{\text{cir}}$  circulant. Applying the FFT then gives:

$$\mathbf{Y} = \mathbf{F} \mathbf{r} = \mathbf{F} \mathbf{H}_{\text{cir}} \mathbf{F}^H \mathbf{X} + \mathbf{F} \nu_{\text{t}} = \text{diag}(\mathbf{H}) \mathbf{X} + \nu, \quad (3.2)$$

the familiar one-tap OFDM model: each active subcarrier satisfies  $Y_k = H_k X_k + \nu_k$ , and post-equalization reliability depends only on accurate estimation of the per-carrier channel gains  $H_k$ . This diagonal baseline is the pristine state of the channel operator. Everything that follows is a perturbation of it.

Table 3.2: More notation used in this chapter.

| Symbol                         | Meaning                                                            | Symbol                                  | Meaning                                                      |
|--------------------------------|--------------------------------------------------------------------|-----------------------------------------|--------------------------------------------------------------|
| $\nu$                          | frequency-domain additive noise vector                             | $\xi_i(\mathbf{z})$                     | relaxed bipolar coded bit, $\tanh z_i$                       |
| $M$                            | number of partial-FFT branches                                     | $s_k(\mathbf{z})$                       | relaxed QPSK symbol induced by two coded bits                |
| $Y_{k,m}$                      | partial-FFT observation on carrier $k$ , branch $m$                | $K_{\text{ICI}}$                        | half-bandwidth of the local ICI model                        |
| $\mathbf{Y}_k$                 | partial-FFT observation vector for carrier $k$                     | $\mathbf{P}$                            | LDPC parity-check matrix                                     |
| $\mathbf{W}_k$                 | combining vector for carrier $k$                                   | $\{\mathcal{N}_j\}$                     | parity-check neighborhoods                                   |
| $\tilde{X}_k$ or $\hat{s}_k$   | pre-decoder scalar soft symbol on carrier $k$                      | $\mathcal{B}_K(k)$                      | local ICI neighborhood around carrier $k$                    |
| $\mathbf{C}$                   | branch-domain local ICI/nonideality parameters                     | $\mathcal{A}_k(\mathbf{C}, \mathbf{s})$ | branch-domain forward model on carrier $k$                   |
| $\mathbf{c}_{k,q}$             | branch response from carrier $q$ to carrier $k$                    | $\lambda_d$                             | weight of the signal term                                    |
| $\Sigma_k$                     | branch-noise covariance for carrier $k$                            | $\eta$                                  | weight of the pilot term                                     |
| $\gamma_C, \gamma_W, \gamma_S$ | weights for branch model, combiner energy, and combiner smoothness | $\rho$                                  | weight tying scalar combiner outputs to relaxed data symbols |
| $\lambda_p$                    | weight of the soft parity-check penalty                            |                                         |                                                              |

### 3.2.2 Packet Time-Scaling Under Doppler

Doppler distortion acts as a time-scaling of the received waveform: if the transmitted packet is  $s(t)$ , the received signal is:

$$r(t) = s((1+a)t),$$

where  $a$  is the Doppler scale. The packet is compressed for  $a > 0$  and stretched for  $a < 0$ , so sample locations shift from the nominal OFDM symbol grid. Coarse Doppler correction reduces the global mismatch but leaves residual intra-symbol variation, which breaks the circulant structure of Equation 3.1: the received OFDM symbol no longer aligns with the nominal FFT basis window.

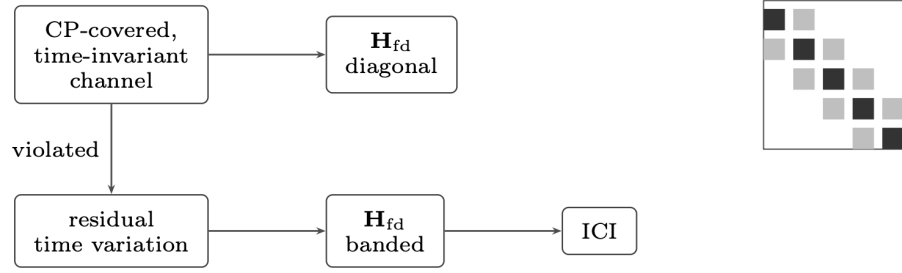


Figure 3.1: Minimal residual-Doppler picture. A CP-covered time-invariant channel is diagonal in the OFDM basis. Residual time variation makes the frequency-domain operator non-diagonal; under local Doppler the dominant off-diagonal terms are near the main diagonal.

### 3.2.3 Loss of Subcarrier Orthogonality

The diagonal model Equation 3.2 required orthogonal subcarriers over the useful interval. Residual Doppler destroys that orthogonality: the waveform no longer matches the nominal basis, and energy leaks between subcarriers. The frequency-domain model becomes:

$$Y_k = H_{k,k}X_k + \sum_{\ell \neq k} H_{k,\ell}X_\ell + \nu_k, \quad (3.3)$$

where the off-diagonal terms  $H_{k,\ell}$  represent Doppler-induced ICI. The receiver must now estimate both the transmitted symbols and how Doppler couples the observations. Figure 3.1 illustrates the transition: the nominal channel is diagonal; residual Doppler causes basis mismatch within the FFT window, off-diagonal leakage in the frequency-domain operator, and visible phase smearing in the received constellation. One-tap pilot-only equalization, which assumes Equation 3.2, becomes unreliable in this regime.

### 3.2.4 Residual Time Variation and Banded Structure

The scalar coupling model Equation 3.3 can be written more precisely as an operator perturbation. Let  $\mathbf{G}$  denote the post-acquisition, CP-removed time-domain channel operator over one useful OFDM block. In the ideal CP-covered time-invariant case,  $\mathbf{G} = \mathbf{H}_{\text{cir}}$  is circulant and the diagonal model Equation 3.2 follows. Under residual time variation,

$$\mathbf{r} = \mathbf{G}\mathbf{F}^H\mathbf{X} + \nu_t, \quad \mathbf{Y} = \mathbf{F}\mathbf{G}\mathbf{F}^H\mathbf{X} + \nu := \mathbf{H}_{\text{fd}}\mathbf{X} + \nu.$$

Thus the diagonal OFDM model is not an assumption about OFDM alone; it follows from the joint assumptions of CP-covered delay spread, correct timing, and channel invariance over the

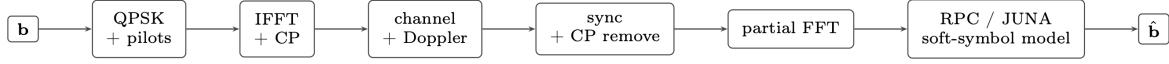


Figure 3.2: Minimal packet-processing chain. The receiver-specific contribution begins after synchronization and CP removal, where partial-FFT observations are passed to the soft-symbol estimator and LDPC-coupled detector.

useful interval. Residual Doppler violates the last condition, so  $\mathbf{G}$  is generally non-circulant and  $\mathbf{H}_{\text{fd}}$  is generally non-diagonal.

For local Doppler distortions, the off-diagonal energy of  $\mathbf{H}_{\text{fd}}$  is usually concentrated near the main diagonal. We therefore use the banded approximation:

$$Y_k = H_{k,k}X_k + \sum_{q \in \mathcal{B}_K(k) \setminus \{k\}} H_{k,q}X_q + \nu_k, \quad \mathcal{B}_K(k) = \{q \in \mathcal{K} : |q - k| \leq K_{\text{ICI}}\}. \quad (3.4)$$

The truncation error is:

$$\epsilon_k^{(K)} = \sum_{q \notin \mathcal{B}_K(k)} H_{k,q}X_q, \quad \mathbb{E}|\epsilon_k^{(K)}|^2 = \sum_{q \notin \mathcal{B}_K(k)} |H_{k,q}|^2 \mathbb{E}|X_q|^2,$$

for independent zero-mean symbols. Increasing  $K_{\text{ICI}}$  reduces model bias but increases the number of front-end coefficients to estimate. Figure 3.1 summarizes this operator-level transition, and Figure 3.2 situates the receiver blocks affected by the loss of diagonality.

### 3.3 Conventional Defenses Against ICI

Residual Doppler leaves a banded operator Equation 3.4 in place of the diagonal baseline Equation 3.2. This section surveys three layers of conventional response to that breakdown — comb pilots for channel anchoring, partial-FFT reception for richer observations, and the RLS-based Partial-FFT Combiner (RPC) for pilot-trained soft-symbol formation — and identifies the point at which separate RPC and decoding begin to hurt under sparse pilots and strong Doppler. That limit is what JUNA is designed to address.

#### 3.3.1 Pilot Anchors and Comb Placement

Active subcarriers are assigned as pilot, data, or null. A *comb pilot pattern* places pilots at regular intervals (every  $S$ -th subcarrier), giving  $N_{\text{pil}} = N_{\text{act}}/S$  pilot tones. Density trades estimation accuracy against payload capacity: denser pilots track Doppler better but cost throughput. The comb placement, illustrated in Figure 3.3, is used throughout the remainder of the report.

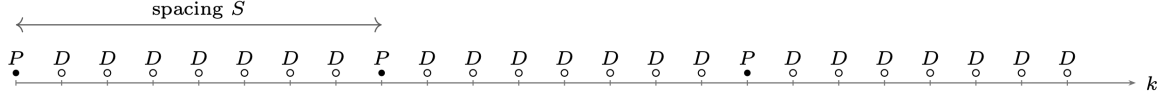


Figure 3.3: Comb pilot pattern. Known pilots are inserted every  $S$  active carriers; intervening carriers carry coded QPSK data.

On a pilot tone  $k \in \mathcal{P}$  with known symbol  $P_k$ , the pilot least-squares estimate of the local diagonal gain is  $\hat{H}_k^{\text{LS}} = Y_k/P_k$ , unbiased with variance  $\sigma^2/|P_k|^2$ . Data-tone estimates are obtained by interpolation from nearby pilots. This interpolation-based picture assumes a diagonal operator and therefore breaks under the off-diagonal leakage of Equation 3.3: one tap per carrier is simply not enough degrees of freedom to represent a banded response. The natural next step is to give the receiver more observations per carrier.

### 3.3.2 Temporal Slicing: Partial-FFT Reception

A conventional receiver represents each active carrier by a single FFT output. Partial-FFT reception instead decomposes the useful OFDM interval into  $M$  temporal components and computes one FFT contribution per component. Let  $g_m[n]$ ,  $m = 1, \dots, M$ , be deterministic segment windows and define:

$$\mathbf{S}_m = \text{diag}(g_m[0], \dots, g_m[N-1]).$$

The branch observation for carrier  $k$  is:

$$Y_{k,m} = \mathbf{e}_k^T \mathbf{F} \mathbf{S}_m \mathbf{r}, \quad \mathbf{Y}_k = [Y_{k,1}, \dots, Y_{k,M}]^T \in \mathbb{C}^M.$$

Equivalently, in sample notation,

$$Y_{k,m} = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} g_m[n] r[n] e^{-j2\pi kn/N}.$$

If the windows form a partition of unity,  $\sum_{m=1}^M g_m[n] = 1$  for every  $n$ , then the full FFT statistic is recovered exactly by summing the branches:

$$\sum_{m=1}^M Y_{k,m} = \mathbf{e}_k^T \mathbf{F} \left( \sum_{m=1}^M \mathbf{S}_m \right) \mathbf{r} = \mathbf{e}_k^T \mathbf{F} \mathbf{r} = Y_k^{\text{full}}.$$

Thus partial-FFT processing does not invent a new measurement; it decomposes the full FFT statistic into temporally localized components that retain information otherwise collapsed by the full FFT.

A useful local interpretation is that residual Doppler mainly introduces slow intra-symbol phase drift. Partitioning the useful interval into segment index sets  $\mathcal{T}_m = \{(m-1)N/M, \dots, mN/M-1\}$  and approximating the channel as fixed across the symbol while an additional phase evolves segment-to-segment gives:

$$r[n] \approx e^{j\phi_m} \sum_{\ell=0}^{L-1} h_\ell x[(n-\ell) \bmod N] + \nu[n], \quad n \in \mathcal{T}_m.$$

Each branch then obeys:

$$Y_{k,m} \approx e^{j\phi_m} H_k X_k + I_{k,m} + \nu_{k,m},$$

where  $I_{k,m}$  collects residual local ICI neglected by this approximation. The branch-level representation makes a concrete question available to the receiver: given the  $M$  branch observations for carrier  $k$ , what linear combination recovers a reliable pre-decoder soft symbol?

### 3.3.3 The RLS-Based Partial-FFT Combiner

The direct answer is to form the pre-decoder soft symbol as:

$$\tilde{X}_k = \mathbf{W}_k^H \mathbf{Y}_k,$$

with  $\mathbf{W}_k \in \mathbb{C}^M$  a carrier-specific combining vector. For a pilot carrier  $p$ , the receiver chooses weights that rotate and scale the branch observations  $\mathbf{Y}_p$  back toward the known pilot symbol  $X_p$ . The combiner is not estimating a full dense ICI matrix but a low-dimensional coherent recombination rule for the partial-FFT branches, following .

Let  $\mathcal{P}(k) \subseteq \mathcal{P}$  be the nearby pilot set for target carrier  $k$ . A pilot-trained ridge least-squares criterion is:

$$J_k(\mathbf{W}_k) = \sum_{p \in \mathcal{P}(k)} |\mathbf{W}_k^H \mathbf{Y}_p - X_p|^2 + \delta \|\mathbf{W}_k\|_2^2, \quad (3.5)$$

with ridge parameter  $\delta > 0$ . Stacking  $\mathcal{P}(k) = \{p_1, \dots, p_{P_k}\}$  into:

$$\mathbf{U}_k = [\mathbf{Y}_{p_1}^H; \dots; \mathbf{Y}_{p_{P_k}}^H], \quad \mathbf{d}_k = [X_{p_1}; \dots; X_{p_{P_k}}].$$

yields  $J_k = \|\mathbf{U}_k \mathbf{W}_k - \mathbf{d}_k\|_2^2 + \delta \|\mathbf{W}_k\|_2^2$  with normal equations:

$$(\mathbf{U}_k^H \mathbf{U}_k + \delta \mathbf{I}) \mathbf{W}_k = \mathbf{U}_k^H \mathbf{d}_k. \quad (3.6)$$

Each pilot contributes one row built from its  $M$  branch observations, so the  $M$ -tap combiner is recovered by a ridge-regularized matrix inversion. Defining  $\mathbf{R}_k = \sum_{p \in \mathcal{P}(k)} \mathbf{Y}_p \mathbf{Y}_p^H + \delta \mathbf{I}$  and  $\mathbf{r}_k = \sum_{p \in \mathcal{P}(k)} \mathbf{Y}_p X_p^*$ , the batch solution is:

$$\widehat{\mathbf{W}}_k = \mathbf{R}_k^{-1} \mathbf{r}_k.$$

The same criterion admits a recursive least-squares update. For pilot regressors  $\mathbf{u}_t = \mathbf{Y}_{p_t}$ , desired responses  $d_t = X_{p_t}$ , and forgetting factor  $\beta \in (0, 1]$ ,

$$\mathbf{g}_t = \frac{\mathbf{P}_{t-1}\mathbf{u}_t}{\beta + \mathbf{u}_t^H \mathbf{P}_{t-1} \mathbf{u}_t}, \quad e_t = d_t - \mathbf{W}_{k,t-1}^H \mathbf{u}_t,$$

$$\mathbf{W}_{k,t} = \mathbf{W}_{k,t-1} + \mathbf{g}_t e_t^*, \quad \mathbf{P}_t = \beta^{-1} (\mathbf{P}_{t-1} - \mathbf{g}_t \mathbf{u}_t^H \mathbf{P}_{t-1}).$$

RLS is the sequential implementation of Equation 3.6, updating an inverse correlation matrix as pilot rows arrive rather than rebuilding the batch matrix. This defines the *RLS-based Partial-FFT Combiner* (RPC), the strongest separate RPC-and-decoding baseline considered here: it uses the same partial-FFT observation structure as JUNA, but remains fundamentally pre-decoder — pilots train the combining coefficients, and LDPC decoding follows afterward.

### 3.3.4 Where Separate RPC and Decoding Fall Short

RPC combines  $M$  partial-FFT observations per carrier, yet its inferential support remains local: each  $\mathbf{W}_k$  is constrained only by nearby pilots or by decision-directed recursion over tentative symbols. Two failure modes follow directly.

First, under sparse pilots the local set  $\mathcal{P}(k)$  can be too small to specify a stable  $M$ -tap combiner; ridge regularization trades bias for variance but cannot manufacture information that is not there. Second, under strong Doppler, decision-directed recursion can feed soft-symbol errors back into subsequent combiner updates: an unreliable pre-decoder soft symbol produces an unreliable tentative decision, which in turn corrupts the next  $\mathbf{W}_k$ .

Both failure modes share the same structural cause. The LDPC decoder carries rich global constraints on the codeword — parity-check equations that couple distant carriers — yet in a separate RPC-and-decoding receiver those constraints are invoked only after soft-symbol formation is complete. When the pilots cannot anchor  $\mathbf{W}_k$  on their own, the code constraints that *could* provide additional anchoring arrive too late. The natural remedy is to let the code participate in soft-symbol formation, not only in post-formation decoding. This is what JUNA does.

## 3.4 JUNA: Coupled Estimation and Code-Constrained Detection

JUNA makes two moves. It replaces the pilot-stacked desired response of Equation 3.5 with a structured packet-level forward model, and it adds a differentiable LDPC parity penalty that regularizes symbol decisions across the entire codeword. The result is a single optimization problem over combining vectors, local nonideality parameters, and latent symbol variables. Before writing it down we fix the notation for LDPC codes and the soft parity surrogate.



### 3.4.1 LDPC Codes, Bit Latents, and QPSK Symbols

Low-density parity-check (LDPC) codes are linear block codes specified by a sparse binary parity-check matrix  $\mathbf{P} \in \mathbb{F}_2^{m \times n_b}$ , where  $n_b$  is the coded-bit length and  $m$  is the number of parity constraints. (We write  $\mathbf{P}$  rather than  $\mathbf{H}$  to avoid conflict with channel notation.) An encoder maps  $\mathbf{u} \in \mathbb{F}_2^k$  to a codeword  $\mathbf{b} \in \mathbb{F}_2^{n_b}$  satisfying:

$$\mathbf{P}\mathbf{b}^T = \mathbf{0} \quad \text{over } \mathbb{F}_2.$$

Belief propagation (BP) on the Tanner graph induced by  $\mathbf{P}$  remains the final hard-decision decoder.

Because the code constraints are binary while the transmitted OFDM symbols are QPSK, the bit-level variables and subcarrier symbols must be separated. Let  $\xi_i \in [-1, 1]$  denote the relaxed bipolar variable associated with coded bit  $b_i$ , where hard symbols satisfy  $\xi_i = 1 - 2b_i$ . We parameterize:

$$\xi_i(\mathbf{z}) = \tanh z_i, \quad z_i \in \mathbb{R}. \quad (3.7)$$

For Gray-labeled QPSK, data carrier  $k$  carries two coded bits  $i_1(k)$  and  $i_2(k)$ , and its relaxed complex symbol is:

$$s_k(\mathbf{z}) = \frac{1}{\sqrt{2}} \left( \xi_{i_1(k)}(\mathbf{z}) + j \xi_{i_2(k)}(\mathbf{z}) \right). \quad (3.8)$$

Pilot carriers use fixed known symbols  $s_p = P_p$ . For parity-check neighborhoods  $\{\mathcal{N}_j\}$ , the differentiable parity penalty is applied to the relaxed bit variables, not directly to complex QPSK symbols:

$$\mathcal{R}_{\text{par}}(\mathbf{z}) = \sum_{j=1}^m \left( 1 - \prod_{i \in \mathcal{N}_j} \xi_i(\mathbf{z}) \right)^2. \quad (3.9)$$

For hard bipolar variables  $\xi_i \in \{+1, -1\}$ , the product equals +1 if and only if the corresponding parity check is satisfied over  $\mathbb{F}_2$ . This gives a smooth DFEC-style code regularizer while preserving the distinction between binary code constraints and complex OFDM symbols.

### 3.4.2 The JUNA Objective

Let  $\mathbf{C} = \{\mathbf{c}_{k,q} : q \in \mathcal{B}_K(k), k \in \mathcal{K}\}$  denote branch-domain ICI coefficients, where  $\mathbf{c}_{k,q} \in \mathbb{C}^M$  is the vector response from transmitted carrier  $q$  to received carrier  $k$ . Let  $\mathbf{w}_k \in \mathbb{C}^M$  be the combiner used to form a scalar soft observation:

$$\hat{s}_k = \mathbf{w}_k^H \mathbf{Y}_k.$$

The branch-domain banded forward model is:

$$\mathcal{A}_k(\mathbf{C}, \mathbf{z}) = \sum_{q \in \mathcal{B}_K(k)} \mathbf{c}_{k,q} s_q(\mathbf{z}), \quad \mathbf{Y}_k = \mathcal{A}_k(\mathbf{C}, \mathbf{z}) + \mathbf{n}_k. \quad (3.10)$$

Here  $s_q(\mathbf{z})$  is given by Equation 3.8 on data carriers and by the known pilot value  $P_q$  on pilot carriers. Setting  $K_{\text{ICI}} = 0$  recovers a per-carrier partial-FFT model; larger  $K_{\text{ICI}}$  captures stronger local mixing while keeping the model banded rather than fully dense.

A dimensionally consistent JUNA objective is:

$$\begin{aligned} \min_{\mathbf{C}, \mathbf{W}, \mathbf{z}} \mathcal{L}(\mathbf{C}, \mathbf{W}, \mathbf{z}) = & \lambda_d \sum_{k \in \mathcal{K}} \left\| \mathbf{Y}_k - \sum_{q \in \mathcal{B}_K(k)} \mathbf{c}_{k,q} s_q(\mathbf{z}) \right\|_{\Sigma_k^{-1}}^2 \\ & + \eta \sum_{p \in \mathcal{P}} |\mathbf{w}_p^H \mathbf{Y}_p - P_p|^2 + \rho \sum_{k \in \mathcal{D}} |\mathbf{w}_k^H \mathbf{Y}_k - s_k(\mathbf{z})|^2 \\ & + \gamma_C \|\mathbf{C}\|_F^2 + \gamma_W \sum_{k \in \mathcal{K}} \|\mathbf{w}_k\|_2^2 + \gamma_S \sum_{k \in \mathcal{K}} \|\mathbf{w}_{k+1} - \mathbf{w}_k\|_2^2 + \lambda_p \mathcal{R}_{\text{par}}(\mathbf{z}). \end{aligned} \quad (3.11)$$

The first term is a generative observation-consistency term. The second anchors known pilots. The third ties the scalar combiner outputs to the relaxed data symbols. The remaining terms regularize branch-domain model size, combiner energy, frequency smoothness, and LDPC parity.  $\Sigma_k$  is the branch-noise covariance; if it is unavailable,  $\Sigma_k = \sigma^2 \mathbf{I}$  gives the ordinary squared-error version.

### 3.4.3 Relation to RPC

JUNA is the natural lifting of the pilot-stacked inverse problem underlying RPC. Three changes connect the two. First, the pilot-only desired responses in Equation 3.5 are replaced by the branch-domain packet model Equation 3.10, so all carriers contribute to front-end refinement rather than only the pilot set  $\mathcal{P}$ . Second, the unknown data-symbol field is generated by bit latents through Equation 3.7–Equation 3.8, so soft bits and QPSK symbols can improve during inference. Third, coding structure enters through Equation 3.9, so parity checks shape the estimation landscape instead of acting only on the final detector output. Where RPC minimizes pilot-driven soft-symbol error and then hands the result to a decoder, JUNA minimizes a single coupled objective in which the branch model, pilots, combiner outputs, and LDPC constraints regularize one another throughout inference.

## 3.5 Alternating Solver

Objective Equation 3.11 is nonconvex because the branch coefficients  $\mathbf{C}$ , combiners  $\mathbf{W}$ , and bit latents  $\mathbf{z}$  are coupled through  $s_k(\mathbf{z})$ , the branch-domain forward model, and the parity products.

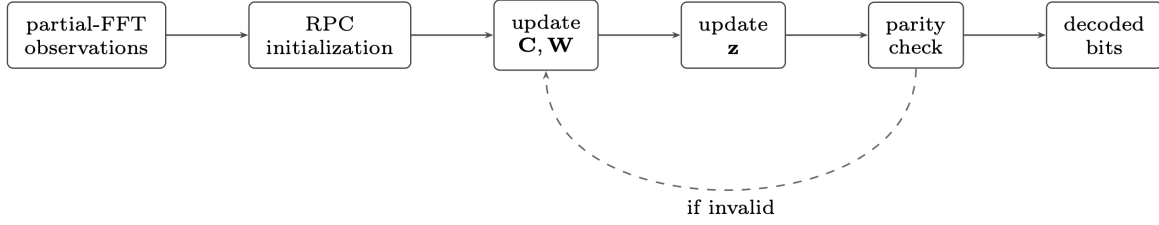


Figure 3.4: Minimal JUNA inference loop. RPC provides the pilot-trained initialization; subsequent iterations alternate front-end and bit-latent updates until a valid parity check or the iteration budget is reached.

The solver therefore does not claim global optimality. It exploits the block structure so that front-end refinement and code-constrained bit refinement support one another throughout the solve rather than acting as separate processing stages (Figure 3.4).

The blocks have different numerical character. For fixed  $\mathbf{z}$ , the update of  $\mathbf{C}$  is a ridge-regularized least-squares problem. For fixed  $(\mathbf{C}, \mathbf{z})$ , the update of each  $\mathbf{w}_k$  is also ridge-regularized least squares. The  $\mathbf{z}$ -block is smooth but nonconvex due to the QPSK map and parity products; gradient or quasi-Newton steps are therefore used as local refinement rather than as a guarantee of global optimality.

### 3.5.1 Soft-Symbol-Model Update

Given current latents  $\mathbf{z}^{(t)}$ , define  $s_k^{(t)} = s_k(\mathbf{z}^{(t)})$ . The front-end subproblem updates  $(\mathbf{C}, \mathbf{W})$  by minimizing the terms of Equation 3.11 that depend on the branch model and scalar combiners:

$$\begin{aligned} \min_{\mathbf{C}, \mathbf{W}} \lambda_d \sum_{k \in \mathcal{K}} \left\| \mathbf{Y}_k - \sum_{q \in \mathcal{B}_K(k)} \mathbf{c}_{k,q} s_q^{(t)} \right\|_{\Sigma_k^{-1}}^2 &+ \eta \sum_{p \in \mathcal{P}} |\mathbf{w}_p^H \mathbf{Y}_p - P_p|^2 \\ &+ \rho \sum_{k \in \mathcal{D}} |\mathbf{w}_k^H \mathbf{Y}_k - s_k^{(t)}|^2 + \gamma_C \|\mathbf{C}\|_F^2 + \gamma_W \sum_k \|\mathbf{w}_k\|_2^2 + \gamma_S \sum_k \|\mathbf{w}_{k+1} - \mathbf{w}_k\|_2^2. \end{aligned}$$

Without the data-symbol and smoothness terms, the  $\mathbf{w}_k$  update reduces to the same pilot-trained ridge system as RPC. Thus RPC is both the strongest separate RPC-and-decoding baseline and a practical initializer/inner block for the joint solver.

For the branch coefficients, define:

$$\mathbf{E}_k = \mathbf{Y}_k - \sum_{q \in \mathcal{B}_K(k)} \mathbf{c}_{k,q} s_q^{(t)}.$$

The resulting  $\mathbf{C}$ -block is linear least squares with ridge regularization. If additional nonlinear nonideality parameters are included, the same term remains differentiable and can be updated

by Wirtinger-gradient steps; the linear banded model is the cleanest version for the current paper.

### 3.5.2 Latent-Bit Update

With  $(\mathbf{C}, \mathbf{W})$  fixed, JUNA updates the bit latents by:

$$\min_{\mathbf{z}} \lambda_d \sum_{k \in \mathcal{K}} \left\| \mathbf{Y}_k - \sum_{q \in \mathcal{B}_K(k)} \mathbf{c}_{k,q} s_q(\mathbf{z}) \right\|_{\Sigma_k^{-1}}^2 + \rho \sum_{k \in \mathcal{D}} |\mathbf{w}_k^H \mathbf{Y}_k - s_k(\mathbf{z})|^2 + \lambda_p \sum_{j=1}^m \left( 1 - \prod_{i \in \mathcal{N}_j} \xi_i(\mathbf{z}) \right)^2.$$

This block is naturally handled by automatic differentiation because  $\mathbf{z} \mapsto \xi_i(\mathbf{z}) \mapsto s_k(\mathbf{z})$  is nonlinear even before the parity products are added. The explicit bit-level notation is important: the parity check acts on  $\xi_i$ , while the OFDM observation model acts on the complex QPSK symbols  $s_k$ .

### 3.5.3 Initialization and Stopping

Because the objective is nonconvex, initialization matters. A natural choice is to seed the soft-symbol model from pilot-trained RPC: initialize  $\mathbf{W}^{(0)}$  from pilot-weighted ridge least squares (or an RLS pass), initialize  $\mathbf{C}^{(0)}$  from a neutral low-order local model, form  $\tilde{s}_k^{(0)} = (\mathbf{w}_k^{(0)})^H \mathbf{Y}_k$ , demap these scalar soft symbols to initial bit latents  $\mathbf{z}^{(0)}$ , and then construct  $s_k(\mathbf{z}^{(0)})$  through Equation 3.8. The joint solver thus starts from a meaningful pilot-trained model before code-coupled iterations begin, and its first outer iteration remains anchored to RPC behavior.

The solver terminates on any of: a valid decode (the most meaningful criterion, as in DFEC), a stalled parity residual, total-loss decrease below a threshold, or exhaustion of the outer-iteration budget.

## 3.6 Validation

The architectural claim of Section 3.4 is sharper than “JUNA reduces BER”. It says JUNA’s advantage *tracks the dynamic non-diagonality of the channel* — large where the residual operator carries heavy off-diagonal energy, small where it does not. That is a statement about which channels benefit, not about how much a single channel improves, and no single SNR sweep can falsify it.

We therefore validate JUNA along two complementary axes. A single-channel SNR sweep on the deterministic six-tap underwater multipath channel used in the RPC literature (Section 3.6.5) pins down an apple-to-apple comparison on the same benchmark against which the strongest

1. Input partial-FFT observations  $\{\mathbf{Y}_k\}$ , pilot set  $\mathcal{P}$ , pilot symbols  $\{X_p\}_{p \in \mathcal{P}}$ , parity-check matrix  $\mathbf{P}$ , weights  $(\lambda_d, \eta, \rho, \gamma_C, \gamma_W, \gamma_S, \lambda_p)$ , and maximum outer iterations  $T_{\max}$ .
2. Initialize  $\mathbf{W}^{(0)}$  from pilot-trained RPC,  $\mathbf{C}^{(0)}$  from a neutral local model, and  $\mathbf{z}^{(0)}$  from initial pre-decoder soft symbols.
3. For  $t = 0, 1, \dots, T_{\max} - 1$ :
4. Update  $\mathbf{W}^{(t+1)}$  by a soft-symbol-model ridge/least-squares step.
5. Update  $\mathbf{C}^{(t+1)}$  by one or more manual-gradient steps.
6. Update  $\mathbf{z}^{(t+1)}$  by one or more AD-based gradient steps.
7. Form hard bit decisions from  $\xi_i(\mathbf{z}^{(t+1)})$  and QPSK symbols from  $s_k(\mathbf{z}^{(t+1)})$ ; stop if all parity checks are satisfied.
8. Return decoded bits and final soft-symbol-model estimates.

Figure 3.5: Algorithm for hybrid JUNA solver.

separate-decoder baseline was originally evaluated. A population study across 60 realistic shallow-water geometries (Section 3.6.6) then tests the architectural prediction directly: do channels with large mean ICI ratio produce the largest RPC/JUNA gains, do channels with little dynamic non-diagonality produce small gains, and does the spread of gains across the cohort match the argument? One study guards against the obvious objection that the result is an artefact of cohort choice; the other tests the prediction the theory actually makes. Both share the packet-level harness described next.

### 3.6.1 Simulation Framework

The packet-level harness is shared between both studies. Each trial runs a complete packet through waveform generation, channel distortion, standard Doppler generation, packet acquisition, partial-FFT observation formation, branch-specific inference, and LDPC-coded error counting.

### 3.6.2 Six-Tap Baseline Channel

The deterministic baseline used for the fixed-channel sweep is the six-tap underwater multipath channel in Table 3.3. This channel is presented first because the RPC reference also uses this style of six-tap underwater acoustic benchmark; keeping it lets us make an apple-to-apple comparison between RPC and JUNA before moving to a larger set of realistic channel geometries. With tap delays  $\{d_\ell\}$ , powers  $\{p_\ell\}$ , and fixed phases  $\{\phi_\ell\}$ ,

$$h[n] = \sum_{\ell=0}^5 \sqrt{p_\ell} e^{j\phi_\ell} \delta[n - d_\ell], \quad H_k = \sum_{\ell=0}^5 \sqrt{p_\ell} e^{j\phi_\ell} e^{-j2\pi k d_\ell / N}.$$

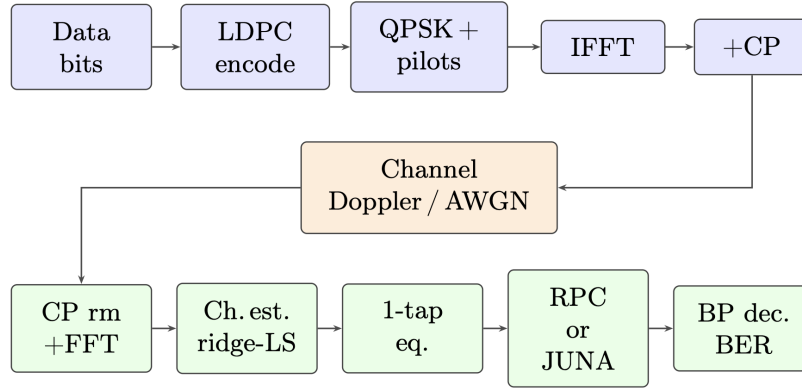


Figure 3.6: Packet-level simulation flow. Data bits are LDPC-encoded, mapped with pilots, converted to CP-OFDM via IFFT, passed through the channel with Doppler and AWGN, then processed by CP removal, pilot ridge-LS channel estimation, one-tap equalization, reception (RPC/JUNA), and BP decoding for BER counting.

Table 3.3: Deterministic six-tap baseline channel used in the fixed-channel benchmark.

| Tap index | Delay (ms) | Power (dB) | Default phase (rad) |
|-----------|------------|------------|---------------------|
| 0         | 0.0        | 0.0        | 0.0                 |
| 1         | 0.8        | -0.9       | 0.7                 |
| 2         | 1.7        | -4.9       | -1.1                |
| 3         | 2.7        | -8.0       | 2.0                 |
| 4         | 3.8        | -7.8       | -2.4                |
| 5         | 5.0        | -23.7      | 1.3                 |

The realistic-channel study then replaces the deterministic taps with impulse responses generated from shallow-water acoustic geometry. The Pekeris model is used here as the propagation method that converts channel parameters — range, bathymetry, transmitter/receiver depths, and boundary conditions — into a channel impulse response. These geometries are a better proxy for real underwater links because they produce a spread of notches, delay tails, and Doppler-induced ICI patterns that a single six-tap channel cannot cover.

### 3.6.3 Simulation Procedure

One trial proceeds as follows: sample data bits, LDPC-encode them, map QPSK data and pilots onto  $N = 2048$  subcarriers, apply the IFFT and a 256-sample CP, pass the packet through the underlying channel and standard Doppler generation, remove the CP after acquisition and Doppler compensation, form the full-FFT and  $P = 4|8|16|32$  partial-FFT observations,

estimate the diagonal channel from pilots using ridge LS, apply one-tap equalization using the estimated channel, run the receiver under test (RPC, or JUNA), and produce hard/soft decisions for BP decoding and error counting. At each SNR, trials repeat until every method accumulates at least 10 coded bit errors or a packet budget is reached. Reported BER is computed from true packet counts via:

$$\text{BER}_m = \frac{\sum_{q=1}^Q e_m^{(q)}}{\sum_{q=1}^Q n_m^{(q)}},$$

where  $e_m^{(q)}$  and  $n_m^{(q)}$  are the coded bit errors and coded bits counted for method  $m$  on packet  $q$ , and  $Q$  is the total number of packets processed at that SNR.

### 3.6.4 Packet and Coding Setup

Each packet is a QPSK payload using  $N = 2048$ , CP length 256, sampling rate  $f_s = 12$  kHz, carrier spacing  $\Delta f = f_s/N = 5.859375$  Hz, and pilot ratio 0.25. At the receiver the packet is distorted by the channel and processed by a coarse-to-fine acquisition that searches over delay and Doppler scale via preamble correlation; after alignment, CP removal, and AWGN addition, the resulting block  $y_{\text{blk}}[n]$  yields:

$$Y_k^{\text{full}} = \frac{1}{\sqrt{N}} \sum_{n=0}^{N-1} y_{\text{blk}}[n] e^{-j2\pi kn/N}, \quad \mathbf{Y}_k = \text{partial\_ffts}(y_{\text{blk}}, P).$$

The diagonal channel used by the one-tap equalizer is estimated from the pilot carriers by a time-domain ridge least-squares fit. Let  $p_r \in \mathcal{P}$  denote the pilot-carrier indices and choose:

$$L_h = \min\{N_{\text{cp}} + 1, \max(8, |\mathcal{P}|\}\}, \quad \rho_{\text{LS}} = 10^{-3}.$$

With:

$$A_{r\ell} = X_{p_r} e^{-j2\pi p_r \ell / N}, \quad \ell = 0, \dots, L_h - 1,$$

the estimator solves:

$$\hat{\mathbf{h}} = \arg \min_{\mathbf{h} \in \mathbb{C}^{L_h}} \sum_{p_r \in \mathcal{P}} \left| Y_{p_r}^{\text{full}} - \sum_{\ell=0}^{L_h-1} A_{r\ell} h_{\ell} \right|^2 + \rho_{\text{LS}} \|\mathbf{h}\|_2^2 = (A^H A + \rho_{\text{LS}} I)^{-1} A^H \mathbf{y}_{\mathcal{P}}. \quad (3.12)$$

The frequency response  $\hat{H}_k$  is obtained by zero-padding  $\hat{\mathbf{h}}$  to length  $N$  and applying an  $N$ -point FFT. The one-tap equalized observation is then:

$$\hat{X}_k = \frac{\hat{H}_k^*}{|\hat{H}_k|^2 + \sigma^2} Y_k^{\text{full}},$$

and the same pilot-estimated channel is used to initialize the separate RPC-and-decoding path and JUNA. For the uniform comb pilot pattern, the implementation evaluates Equation 3.12 with the equivalent FFT-diagonalized computation, which gives the same ridge-LS estimate without forming  $A$  explicitly.

The coded benchmark uses a rate-1/2 LDPC code with BP decoder. The same parity-check matrix is exposed to JUNA via its code penalty, so the joint and post-receiver decoders are tested under identical packet coding assumptions. Table 3.4 lists the remaining parameters.

Table 3.4: Default packet-level benchmark configuration used in the comparison study.

| Item                            | Value                                                                                                                                                                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OFDM size $N$                   | 2048                                                                                                                                                                                                                                                   |
| Cyclic prefix                   | 256 samples                                                                                                                                                                                                                                            |
| Pilot-LS channel length $L_h$   | Automatic, $L_h = \min\{N_{\text{cp}} + 1, \max(8,  \mathcal{P} \})\}$                                                                                                                                                                                 |
| Partial-FFT branches $P$        | 32                                                                                                                                                                                                                                                     |
| Pilot ratio                     | 0.20                                                                                                                                                                                                                                                   |
| Sampling rate / carrier spacing | 12 kHz / 5.859375 Hz                                                                                                                                                                                                                                   |
| FEC                             | Rate-1/2 LDPC code with BP decoder.                                                                                                                                                                                                                    |
| Stopping rule                   | minimum 10 coded errors or maximum 1000 packets per SNR                                                                                                                                                                                                |
| RPC parameters                  | $\lambda = 0.998$ , $\delta = 5$ , $\alpha = 0.05$                                                                                                                                                                                                     |
| JUNA defaults                   | 300 iterations, 4 subbands, $K_{\text{ICI}} = 2$ , pilot-LS init for $\mathbf{W}$ , delta init for $\mathbf{C}$ ,<br>$\lambda_{\text{pil}} = \lambda_{\text{dat}} = \lambda_{\text{code}} = 1$ , $\lambda_{\text{const}} = 0.25$ ,<br>demixing enabled |

### 3.6.5 Packet-Level Verification on the Baseline Channel

Before turning to the population study, we verify on the deterministic six-tap baseline of Table 3.3 that JUNA actually delivers a coded-BER advantage over separate RPC-and-BP on a single channel. This is the apple-to-apple comparison: the same six-tap underwater multipath channel against which the RPC literature is benchmarked, run end-to-end through the packet-level harness of Section 3.6.1.

The executed run used SNR 0:2:10 dB, standard Doppler generation with draws  $a \sim \text{Uniform}[-1.5 \times 10^{-4}, 1.5 \times 10^{-4}]$ , and the stopping rule “min 10 coded errors per method or max 300 packets per SNR.” The 0, 2, 4, and 6 dB points completed during the run (with completed packet counts 1, 1, 4, and 26 respectively); the directly observed values are plotted in Figure 3.7. JUNA outperforms separate RPC and BP decoding over the completed low-to-mid SNR points, with the separation growing from 4 dB onward. This is the



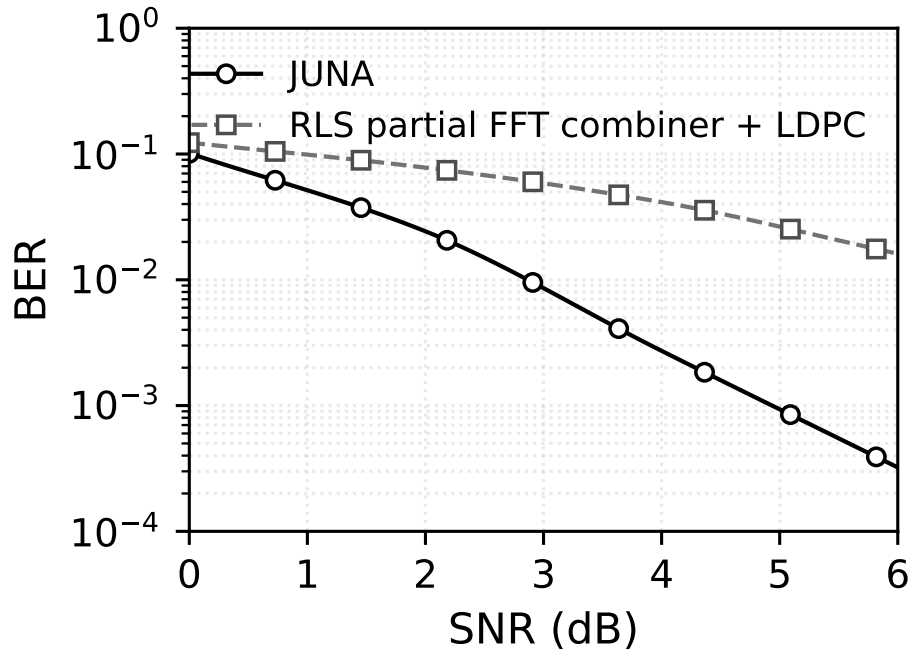


Figure 3.7: Coded BER versus SNR from the directly observed completed points of the executed packet-level sweep on the deterministic six-tap baseline channel.

single-channel pattern that the 60-channel population study below extends to a much wider range of operating conditions.

### 3.6.6 Realistic Channel Geometry Study

One SNR sweep on one channel is not enough. The architectural argument of Section 3.4 predicts *which* channels should benefit most from JUNA — those where residual Doppler creates strong carrier-to-carrier mixing — and a claim of that shape only becomes testable across an ensemble. So we sweep 60 shallow-water geometries instead of one.

The geometries vary along four axes: range (100–500 m), bathymetry (20–50 m), transmitter and receiver depths (5–15 m), and the bottom and surface boundary types (fluid, rigid, pressure-release, windy). Each combination is fed through a Pekeris propagation model, and the impulse response that comes out is used by the simulation flow of Section 3.6.5. The full configuration list is long; it is shown in Table 3.6, where the channels are renumbered 1–60 in ascending library order so the BER figures can use compact IDs.

The rest of the subsection looks at this ensemble through three views at increasing zoom. The error-floor histogram (Figure 3.8) gives the ensemble view. Five landmark channels (Table 3.5

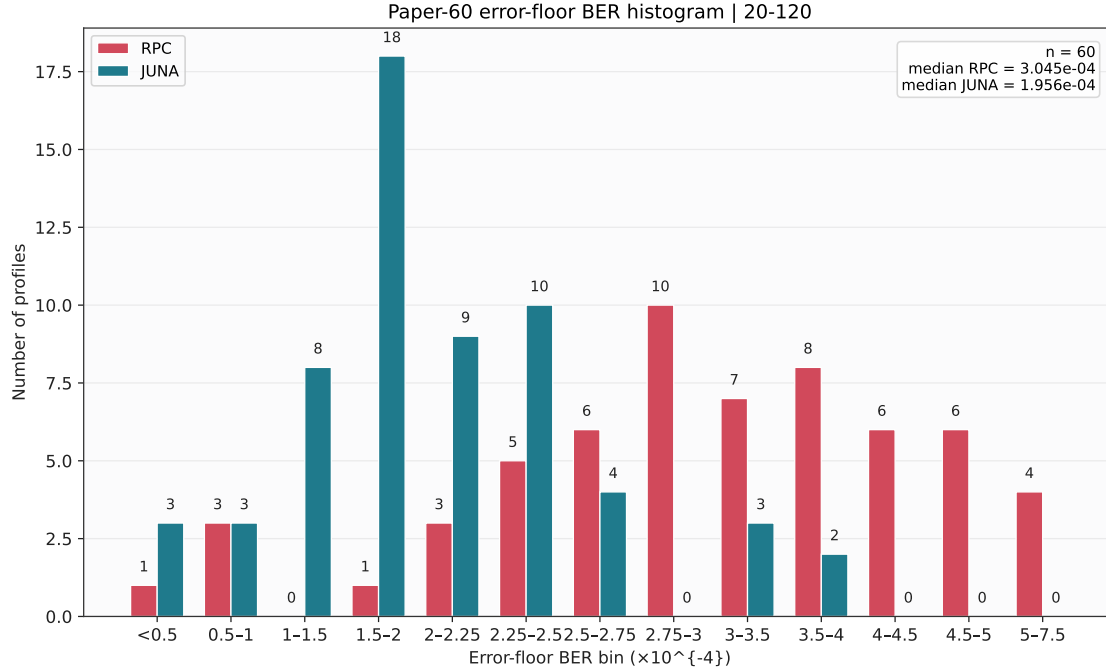


Figure 3.8: Grouped histogram of error-floor BER for the reference 60-channel ensemble. Horizontal axis: error-floor BER, binned in units of  $10^{-4}$ . Vertical axis: number of channels (out of 60) whose error floor falls in each bin. Each bin contains two side-by-side bars; the red one counts channels under separate RPC to BP decoding and the blue one counts channels under JUNA. JUNA mass is concentrated on the left (lower BER), RPC mass on the right (higher BER), matching the per-channel results.

and Figure 3.9) tie that ensemble shift to specific geometries. A pilot-ratio sweep on one channel (Figure 3.10) then exposes the rescue mechanism behind the shift.

### 3.6.7 Ensemble shift: error-floor histogram

Figure 3.8 compresses the 60-channel ensemble into a one-dimensional view. The horizontal axis is the error-floor BER, binned in units of  $10^{-4}$ . The vertical axis counts how many channels fall into each bin. Each bin contains two side-by-side bars: a red one for separate RPC→BP decoding, a blue one for JUNA.

Sweeping left to right, the JUNA bars dominate the low-BER bins, the two methods overlap in the middle, and the RPC bars take over the high-BER tail. The whole distribution shifts. So JUNA’s advantage is not a handful of friendly geometries — it is a property of the cohort as a whole.

### 3.6.8 Five landmark channels

The histogram says the ensemble shifts; it does not say which geometries drive that shift. Table 3.5 picks five landmark channels that span the corners of the cloud, and Figure 3.9 unfolds each one into a four-panel row showing impulse response, static frequency response, and coded BER at  $a = 0$  and at  $a = 10^{-4}$ .

Table 3.5: Selected channels from the 60-channel ensemble. BER(JUNA) and gain values are read at the automatically detected error-floor SNR for  $a = 10^{-4}$ .

| ID | Label                              | Notch (dB) | SFM   | $\bar{\eta}$ | BER(JUNA)             | Gain<br>RPC/JUNA |
|----|------------------------------------|------------|-------|--------------|-----------------------|------------------|
| 42 | flat<br>low-BER<br>gain edge       | 11.68      | 0.782 | 0.0202       | $1.21 \times 10^{-5}$ | 4.09             |
| 20 | high-gain<br>selective<br>edge     | 28.64      | 0.707 | 0.0295       | $1.83 \times 10^{-5}$ | 3.75             |
| 13 | high-gain<br>selective<br>mid-band | 37.08      | 0.594 | 0.0258       | $1.43 \times 10^{-4}$ | 3.25             |
| 60 | selective<br>weak-tail<br>mid-band | 38.42      | 0.577 | 0.0529       | $1.95 \times 10^{-4}$ | 1.30             |
| 52 | high-ICI<br>hard floor             | 39.67      | 0.559 | 0.1409       | $3.63 \times 10^{-4}$ | 0.98             |

Three of the landmarks mark the main regions: an easy low-BER edge (channel 42), a more selective mid-band case (channel 60), and a hard high-ICI floor (channel 52). The other two, channels 20 and 13, sit at the high-gain end of the RPC/JUNA comparison and show that large gains can persist deep into the selective regime. Two summary metrics appear in Table 3.5. *Notch depth* is the worst-case max-to-min frequency-response ratio in dB, and *SFM* is the geometric-to-arithmetic mean of the same response; both track static fragility. The *mean ICI ratio*  $\bar{\eta}$  measures how strongly residual Doppler at  $a = 10^{-4}$  leaks energy off the diagonal of the OFDM operator, and tracks dynamic leakage.

The five rows make the histogram shift concrete. Channel 42 is the easy edge: the flattest response of the five, the smallest  $\bar{\eta}$ , and the lowest JUNA error floor. Even here, the Doppler panel keeps a visible gap between RPC and JUNA. Channels 20 and 13 push the same high-gain story into more selective territory — they are notchier than 42, yet their  $\bar{\eta}$  stays modest, and JUNA turns that asymmetry into a sizable BER gain. Channel 60 is harder. The notches stay deep, the JUNA floor lifts, and the gain shrinks. Channel 52 is the hard wall: largest  $\bar{\eta}$  of the

five, highest JUNA error floor, and RPC/JUNA ratio close to one. That last point is the useful reminder — big mean ICI on its own is not enough to guarantee a big gain.

### 3.6.9 Pilot-ratio rescue on channel 20

The histogram and the five landmarks say *what* JUNA improves; the pilot-ratio sweep says *when* that improvement appears. Figure 3.10 fixes one channel (channel 1, the first row of Table 3.6), holds SNR at 16 dB and residual Doppler at  $a = 10^{-4}$ , and varies the pilot ratio from 0.05 to 0.25 in steps of 0.01.

The two curves break into three regions. Below pilot ratio 0.12, the estimator does not have enough pilots to pin the channel down under residual Doppler; both methods sit at the same high floor near  $10^{-1}$  and adding one more pilot does almost nothing. Between 0.13 and 0.15 the curves drop by roughly two orders of magnitude. This is the rescue threshold, where pilot support finally overcomes the residual-Doppler ambiguity. Above 0.15 both methods plateau, but JUNA's plateau sits roughly half an order of magnitude below RPC's, and that gap holds out to pilot ratio 0.25. The ensemble shift seen in the histogram is the distributional consequence of this single-channel rescue: under JUNA most channels sit to the right of their own threshold, while under separate RPC→BP a non-trivial fraction still sits to the left.

Table 3.6 lists the 60 channel geometries used in the study of Section 3.6. Each row specifies range, bathymetry, transmitter and receiver depths, and the bottom/surface boundary conditions used by the acoustic propagation model.

Table 3.6: Configuration table for the 60 channel geometries. Distances are in meters. Boundary abbreviations: F = fluid, R = rigid, PR = pressure-release, W = windy.

| ID | Rng | Bath | Tx | Rx | Bot. | Surf. | ID | Rng | Bath | Tx | Rx | Bot. | Surf. |
|----|-----|------|----|----|------|-------|----|-----|------|----|----|------|-------|
| 1  | 300 | 30   | 10 | 10 | F    | PR    | 31 | 400 | 20   | 10 | 15 | PR   | R     |
| 2  | 150 | 30   | 10 | 10 | F    | PR    | 32 | 300 | 30   | 15 | 10 | R    | PR    |
| 3  | 200 | 20   | 10 | 15 | R    | R     | 33 | 500 | 50   | 15 | 5  | R    | PR    |
| 4  | 100 | 30   | 5  | 10 | F    | R     | 34 | 200 | 30   | 5  | 15 | R    | PR    |
| 5  | 150 | 30   | 10 | 10 | F    | W     | 35 | 200 | 40   | 15 | 10 | PR   | PR    |
| 6  | 300 | 40   | 10 | 15 | PR   | R     | 36 | 500 | 50   | 5  | 5  | R    | R     |
| 7  | 200 | 30   | 5  | 15 | R    | R     | 37 | 200 | 30   | 10 | 10 | R    | R     |
| 8  | 500 | 40   | 5  | 15 | F    | R     | 38 | 150 | 20   | 10 | 10 | F    | R     |
| 9  | 150 | 30   | 5  | 5  | R    | R     | 39 | 400 | 50   | 5  | 5  | R    | PR    |
| 10 | 400 | 30   | 15 | 5  | F    | W     | 40 | 200 | 40   | 5  | 15 | R    | PR    |
| 11 | 400 | 50   | 5  | 5  | F    | PR    | 41 | 200 | 40   | 15 | 15 | PR   | W     |
| 12 | 500 | 30   | 15 | 15 | R    | W     | 42 | 200 | 30   | 15 | 15 | R    | W     |
| 13 | 500 | 30   | 15 | 15 | PR   | W     | 43 | 200 | 20   | 5  | 5  | F    | R     |
| 14 | 200 | 30   | 15 | 10 | PR   | PR    | 44 | 150 | 30   | 5  | 15 | R    | W     |
| 15 | 500 | 30   | 5  | 5  | F    | R     | 45 | 500 | 20   | 5  | 5  | F    | W     |

| ID | Rng | Bath | Tx | Rx | Bot. | Surf. | ID | Rng | Bath | Tx | Rx | Bot. | Surf. |
|----|-----|------|----|----|------|-------|----|-----|------|----|----|------|-------|
| 16 | 400 | 30   | 10 | 5  | R    | W     | 46 | 200 | 40   | 5  | 5  | R    | R     |
| 17 | 200 | 40   | 5  | 15 | F    | PR    | 47 | 300 | 30   | 5  | 5  | R    | PR    |
| 18 | 400 | 50   | 15 | 5  | PR   | PR    | 48 | 500 | 30   | 5  | 5  | R    | R     |
| 19 | 400 | 30   | 5  | 5  | F    | R     | 49 | 400 | 30   | 15 | 5  | F    | R     |
| 20 | 400 | 30   | 15 | 10 | PR   | R     | 50 | 500 | 40   | 5  | 10 | F    | W     |
| 21 | 100 | 20   | 5  | 10 | F    | R     | 51 | 150 | 40   | 5  | 5  | F    | R     |
| 22 | 150 | 30   | 10 | 10 | R    | PR    | 52 | 100 | 30   | 15 | 10 | PR   | PR    |
| 23 | 400 | 50   | 5  | 5  | R    | R     | 53 | 150 | 30   | 10 | 15 | PR   | R     |
| 24 | 100 | 30   | 10 | 10 | PR   | PR    | 54 | 200 | 40   | 10 | 5  | R    | PR    |
| 25 | 300 | 50   | 10 | 10 | R    | W     | 55 | 200 | 20   | 5  | 5  | R    | PR    |
| 26 | 150 | 40   | 10 | 5  | F    | W     | 56 | 200 | 40   | 5  | 10 | PR   | R     |
| 27 | 400 | 40   | 15 | 10 | PR   | R     | 57 | 150 | 20   | 5  | 15 | PR   | PR    |
| 28 | 150 | 30   | 15 | 5  | PR   | PR    | 58 | 300 | 50   | 5  | 15 | F    | R     |
| 29 | 300 | 20   | 10 | 10 | PR   | R     | 59 | 300 | 30   | 5  | 15 | F    | R     |
| 30 | 200 | 30   | 15 | 5  | F    | R     | 60 | 300 | 40   | 5  | 5  | PR   | R     |

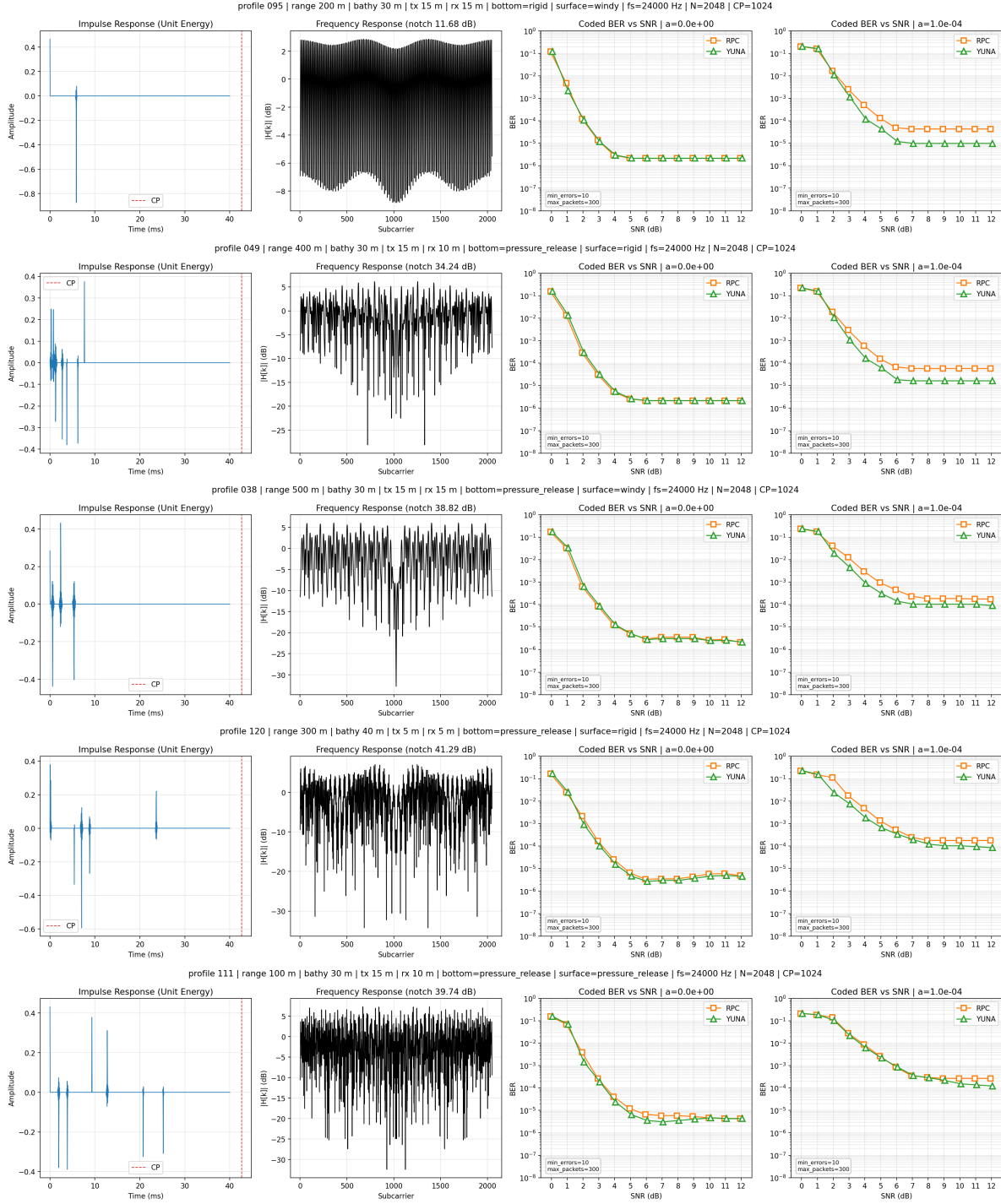


Figure 3.9: Five selected channel geometries arranged as a  $5 \times 4$  panel summary. Each row shows impulse response, static frequency response, coded BER for  $a = 0$ , and coded BER for  $a = 10^{-4}$ . From top to bottom: channel 42 (flat low-BER gain edge), channel 20 (high-gain selective edge), channel 13 (high-gain selective mid-band), channel 60 (selective weak-tail mid-band), and channel 52 (high-ICI hard floor with

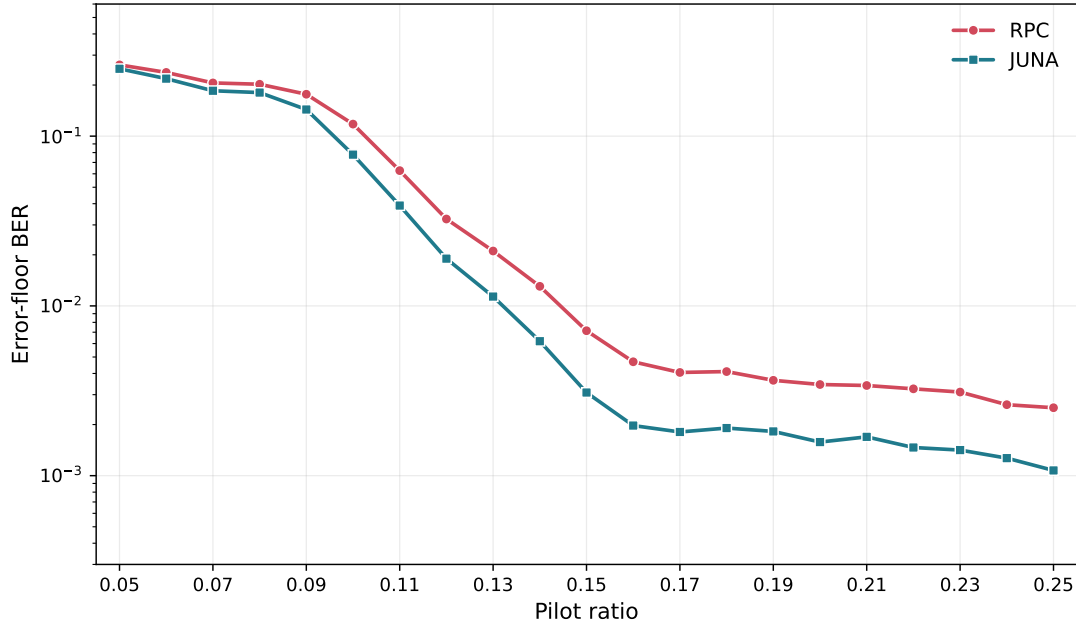


Figure 3.10: Pilot-ratio sweep on channel 1. Horizontal axis: pilot ratio. Vertical axis: error-floor BER at SNR 16 dB and residual Doppler  $a = 10^{-4}$ , shown on a log scale. The red curve is separate RPC to BP decoding; the blue curve is JUNA. Both methods sit at a high floor for pilot ratios below about 0.13, then drop by roughly two orders of magnitude over 0.13 to 0.15. Above the threshold, JUNA remains consistently below RPC.

## 4 Summary and Next Steps

In this deliverable, we reported on major threads of work:

1. The work on adaptive communication in Chapter 2 presented two key components:
  - The first component focussed on the integration of adaptive communication into the protocol stack used in Subnero modems. Getting this right is key to a seamless user experience, where the algorithms developed in this project will provide good performance to the user without him having to explicitly do anything.
  - The second component focussed on the development of a physics-based performance model and recommendation algorithm that can glean information from ongoing communication traffic, and make OFDM parameter recommendations without any protocol overhead. Both were tested using channel replay models from experiments in Singapore waters.
2. The work on integrating DFEC into the communication stack in Chapter 3. Since our modems use OFDM and LDPC as the primary *workhorse* techniques for high-speed communications, we focused our efforts on using DFEC into the OFDM + LDPC processing chain. The result is an algorithm we dubbed JUNA, in which soft-symbol estimates from LDPC are processed together with OFDM pilots to perform joint channel and bit estimation. The results are promising. When the SNR is favorable, the error floor using JUNA is lower than that under traditional methods. This would translate to a better packet success rate in channel conditions where the SNR is high but there is motion and time-variability – exactly the conditions where traditional OFDM struggles.

While the progress so far has been great, much work still remains. The adaptive communication algorithm needs to be tested on a larger set of replay channels, fine tuned, and then integrated into the Subnero modems. It can then be tested using VAO, and then finally it has to be tested at sea. The DFEC integration into OFDM/LDPC is still in early stages of development, with a primary focus on simulation studies to understand its performance. We next need to test it with channel replay to see how it might perform in realistic conditions, and then tune its computational complexity to see if it can be implemented on Subnero modems. Once we have the final version that a modem can run, we need to integrate it into UnetStack and test at sea.

The next 9 months leading up to deliverable D2 will still focus on theoretical development and replay testing, but with more focus on computational complexity and implementation considerations on Subnero modems. The last 6 months of the project will then focus on



integration and in-field testing, the results of which will be summarized in the final deliverable (D3).